

Table of Contents

1. Team Member:	3
2. UVM testbench drawing	4
3. UVM Verification Architecture Analysis for SPI Wrapper Project	4
a. Overview	4
b. Test Hierarchy and Top Module	5
c. Wrapper Environment	5
d. SPI Slave Environment	6
e. RAM Environment	6
f. Data Flow & Test Behavior	7
g. Verification Objectives	7
h. Advantages of This Hierarchical Approach	7
i. Summary	7
4. Code Snippets:	8
a. Part 1: UVM Environment for SPI-Slave	8
1. Verification Plan	8
2. SPI_slave design:	8
3. SPI_slave golden design:	11
4. SPI_slave interface:	13
5. SPI_slave config:	13
6. SPI_slave seq item:	14
7. SPI_slave main seq:	16
8. SPI_slave reset seq:	17
9. SPI_slave driver:	17
10. SPI_slave monitor:	18
11. SPI_slave_sequencer:	19
12. SPI_slave_agent:	19
13. SPI_slave_coverage	20
14. SPI_slave_scoreboard:	21
15. SPI_Slave env:	22
16. SPI_slave_test:	23
17. SPI_slave top:	24
18. Code Coverage:	25

19.	Assertions:	25
20.	Waveform:	26
21.	Assertions Table	26
22.	Bug report	27
b.	Part 2: UVM Environment for Single-Port RAM	28
1.	Verification Plan	28
2.	Assertions of RAM:	29
3.	Configuration object of RAM:	31
4.	Interface of RAM:	31
5.	Top of RAM:	32
6.	Test of RAM:	32
7.	Environment of RAM:	34
8.	Agent of RAM:	35
9.	Sequence of RAM:	37
10.	Sequence item of RAM:	41
11.	Sequencer of RAM:	43
12.	Coverage collector of RAM:	43
13.	Driver of RAM:	46
14.	Monitor of RAM:	47
15.	Scoreboard of RAM:	48
16.	Do file of RAM:	52
17.	Simulation of RAM:	52
18.	Transcripts:	53
19.	Report on RAM:	55
20.	Assertions Table	59
21.	Bug Report:	59
c.	Part 3: UVM Environment for SPI Wrapper	60
1.	Verification Plan	60
2.	Top Module	61
3.	Wrapper Interface	62

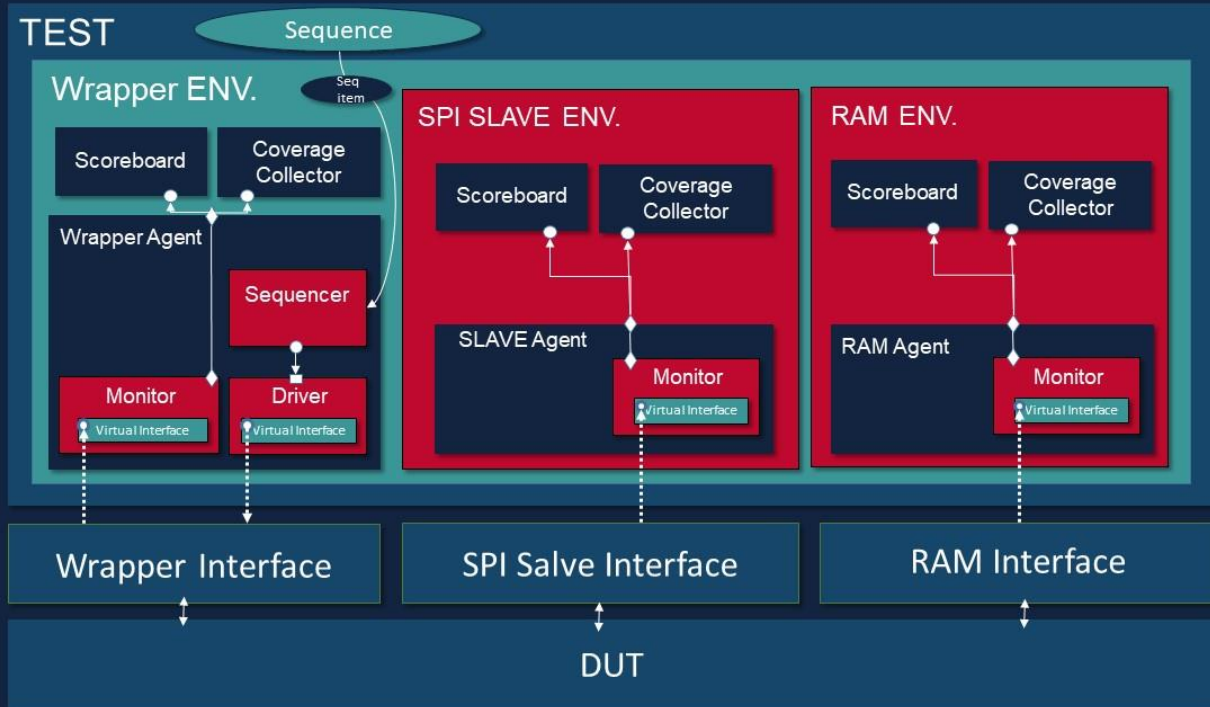
4.	Wrapper Assertions	62
5.	Test Code	63
6.	Wrapper Environment	64
7.	Wrapper Configuration object.....	65
8.	Wrapper Sequence Item	65
9.	Wrapper Sequencer.....	68
10.	Wrapper Agent	68
11.	Wrapper Driver.....	69
12.	Wrapper Monitor	70
13.	Wrapper Scoreboard.....	71
14.	Wrapper Coverage Collector.....	72
15.	Coverage.....	73
16.	QuestaSim Waveforms.....	74
17.	Assertions table	75

■ Team Member:

Name	Group Num:
Mina Hakim	Group 2
Hanya Ibrahim Aref	Group 2
George Safwat Wasfy	Group 1

UVM testbench drawing

TOP Module



UVM Verification Architecture Analysis for SPI Wrapper Project

a. Overview

The diagram illustrates the **UVM-based testbench** developed to verify an SPI Wrapper module that integrates two key submodules:

- An **SPI Slave**, which communicates with an external SPI master through SPI signals (MOSI, MISO, SS_n, and SCLK).
- A **RAM**, which stores or retrieves data based on SPI transactions.

To validate this system, the testbench is organized into **three distinct UVM environments**:

1. **Wrapper Environment (Wrapper ENV.)**
2. **SPI Slave Environment (SPI SLAVE ENV.)**
3. **RAM Environment (RAM ENV.)**

Each environment encapsulates its own **UVM agent(s), monitor(s), scoreboard, and coverage collector**, ensuring modularity and separation of concerns.

b. Test Hierarchy and Top Module

At the **top level**, the `TEST` component instantiates:

- The three environments (`Wrapper ENV.`, `SPI SLAVE ENV.`, and `RAM ENV.`)
- The `Sequence` and `Sequence Item` classes that drive stimulus into the DUT via the Wrapper driver.

Below the testbench, the **DUT (Device Under Test)** — the SPI Wrapper — connects to three interfaces:

- **Wrapper Interface**
- **SPI Slave Interface**
- **RAM Interface**

These interfaces act as connection bridges between the DUT and the UVM testbench via **virtual interfaces**.

c. Wrapper Environment

The **Wrapper Environment** is the **main control environment** that generates test stimulus for the DUT. It contains the following components:

- **Sequencer:**
Generates and controls the flow of `sequence_items` representing SPI wrapper transactions (e.g., read/write commands, address, data).
- **Driver:**
Receives sequence items from the sequencer and drives the corresponding signal activity onto the **Wrapper Interface**.
The driver converts high-level transactions into pin-level activity such as toggling `SS_n`, sending MOSI data, and managing clock synchronization.
- **Monitor:**
Observes the Wrapper Interface signals and sends monitored transactions to the scoreboard and coverage collector for checking.
- **Scoreboard and Coverage Collector:**
 - **Scoreboard:** Compares expected behavior (predicted transactions) against actual DUT output.
 - **Coverage Collector:** Records functional coverage metrics, ensuring all address ranges, commands, and data patterns are exercised.

d. SPI Slave Environment

The **SPI Slave Environment** focuses on verifying the internal SPI slave module behavior within the wrapper.

Its components include:

- **Slave Agent:**
A passive UVM agent containing a **Monitor** that observes the SPI slave interface signals such as MOSI, MISO, SCLK, and SS_n.
- **Monitor:**
Captures SPI frame-level transactions and reconstructs the transmitted or received data. These observations are forwarded to:
 - The **Scoreboard** (for checking correctness of SPI communication)
 - The **Coverage Collector** (for SPI command, address, and data pattern coverage)
- **Scoreboard & Coverage Collector:**
Validate that the SPI Slave correctly interprets SPI transactions from the Wrapper and outputs the expected responses on MISO, without protocol violations or timing errors.

e. RAM Environment

The **RAM Environment** verifies the correctness of memory operations triggered by the SPI Wrapper's internal logic.

- **RAM Agent:**
Passively monitors read and write transactions occurring between the SPI Wrapper and the RAM module via the **RAM Interface**.
- **Monitor:**
Detects data and address lines during memory operations and reconstructs high-level read/write transactions.
- **Scoreboard:**
Compares expected memory contents (predicted by the Wrapper's transactions) with the actual data observed in the RAM interface.
Any mismatch indicates a failure in address decoding, data transfer, or timing control.
- **Coverage Collector:**
Tracks how many different memory locations were accessed, how many reads/writes occurred, and ensures complete verification of the memory-mapped region.

f. Data Flow & Test Behavior

1. The **test class** initializes all environments and sequences.
2. The **Wrapper Sequence** generates a stream of SPI wrapper transactions (read/write).
3. The **Wrapper Driver** drives these transactions onto the Wrapper Interface, which controls the DUT's SPI wrapper.
4. The DUT's SPI wrapper:
 - Forwards SPI commands to the **SPI Slave** interface.
 - Interacts with the **RAM** through internal address and data buses.
5. The **SPI Slave Monitor** and **RAM Monitor** observe these interactions simultaneously.
6. The **Scoreboards** in all environments cross-check the expected vs. actual behavior:
 - Wrapper scoreboard ensures the end-to-end transaction result is correct.
 - SPI Slave scoreboard validates SPI protocol compliance.
 - RAM scoreboard ensures correct memory read/write operations.
7. **Coverage Collectors** record which transactions, commands, and data patterns were tested to measure functional coverage.

g. Verification Objectives

This testbench architecture aims to verify:

- Correct SPI transaction decoding and protocol handling by the SPI Slave.
- Accurate address decoding and data mapping between SPI Wrapper and RAM.
- Stability and correctness of the MISO/MOSI signals.
- No data corruption or timing violations during transactions.
- Full coverage of SPI commands, data sizes, and address ranges.

h. Advantages of This Hierarchical Approach

- **Modularity:** Each environment can be reused or independently debugged.
- **Scalability:** Future modules (e.g., additional SPI slaves or memory types) can be added easily.
- **Separation of Concerns:** Each environment checks one functional layer (Wrapper, SPI, or RAM).
- **Reusability:** The SPI Slave and RAM Environments can be reused in other verification projects.

i. Summary

This UVM testbench implements a **layered and modular verification strategy** for the SPI Wrapper design. The coordinated operation of the **Wrapper**, **SPI Slave**, and **RAM environments** ensures comprehensive verification at:

- **Protocol level** (SPI correctness)
- **Functional level** (Wrapper logic)
- **Data integrity level** (RAM access correctness)

This structure guarantees both **functional accuracy** and **robustness** of the SPI Wrapper DUT under all possible operating scenarios.

Code Snippets:

a. Part 1: UVM Environment for SPI-Slave

1. Verification Plan

Label	Design description	Functional coverage	Function check	stimulus generation
Reset check	When the reset signal is activated, the MISO,rx_valid,rx_data should be zero	Covering the transition of the SS_n after 13 cycle except for the READ_DATA would be for 23 cycle	In the scoreboard , comparing the outputs with the expected of a golden model also using assertions in the sva file	direct reset assertions using reset sequence
IDLE	When the reset signal is deactivated and the SS_n is high , the cs should be IDLE	Covering the transition of the SS_n after 13 cycle except for the READ_DATA would be for 23 cycle	In the scoreboard , comparing the outputs with the expected of a golden model also using assertions in the sva file	randomizing the SS_n and constraining it to be low for 13 cycle for all cases except for the READ_DATA the SS_n would be low for 23 cycle
CHK_CMD	When the reset signal is deactivated and the SS_n is low and the cs is IDLE,the cs should be CHK_CMD	Covering the transition of the MOSI to cover the sequence of the WRITE_ADDR , WRITE_DATA,READ_ADDR,READ_DATA and crossing it with the SS_n coverpoint.	In the scoreboard , comparing the outputs with the expected of a golden model also using assertions in the sva file	randomizing the SS_n and constraining it to be low for 13 cycle for all cases except for the READ_DATA the SS_n would be low for 23 cycle
WRITE_ADDR	When the reset signal is deactivated and the SS_n is low and the cs is CHK_CMD and the MOSI is taking the following sequence 3'b000, the cs should be WRITE_ADDR	Covering the transition of the MOSI to cover the sequence of the WRITE_ADDR , WRITE_DATA,READ_ADDR,READ_DATA and crossing it with the SS_n coverpoint.	In the scoreboard , comparing the outputs with the expected of a golden model	randomizing the SS_n and constraining it to be low for 13 cycle for all cases except for the READ_DATA the SS_n would be low for 23 cycle
WRITE_DATA	When the reset signal is deactivated and the SS_n is low and the cs is CHK_CMD and the MOSI is taking the following sequence 3'b001, the cs should be WRITE_DATA	Covering the transition of the MOSI to cover the sequence of the WRITE_ADDR , WRITE_DATA,READ_ADDR,READ_DATA and crossing it with the SS_n coverpoint.	In the scoreboard , comparing the outputs with the expected of a golden model	randomizing the SS_n and constraining it to be low for 13 cycle for all cases except for the READ_DATA the SS_n would be low for 23 cycle
READ_ADDR	When the reset signal is deactivated and the SS_n is low and the cs is CHK_CMD and the MOSI is taking the following sequence 3'b010, the cs should be READ_ADDR	Covering the transition of the MOSI to cover the sequence of the WRITE_ADDR , WRITE_DATA,READ_ADDR,READ_DATA and crossing it with the SS_n coverpoint.	In the scoreboard , comparing the outputs with the expected of a golden model	randomizing the SS_n and constraining it to be low for 13 cycle for all cases except for the READ_DATA the SS_n would be low for 23 cycle
READ_DATA	When the reset signal is deactivated and the SS_n is low and the cs is CHK_CMD and the MOSI is taking the following sequence 3'b111, the cs should be READ_DATA	Covering the transition of the MOSI to cover the sequence of the WRITE_ADDR , WRITE_DATA,READ_ADDR,READ_DATA and crossing it with the SS_n coverpoint.	In the scoreboard , comparing the outputs with the expected of a golden model	randomizing the SS_n and constraining it to be low for 13 cycle for all cases except for the READ_DATA the SS_n would be low for 23 cycle

```

module SLAVE (MOSI,MISO,SS_n,clk,rst_n,rx_data,rx_valid,tx_data,tx_valid);

    localparam IDLE      = 3'b000;
    localparam CHK_CMD   = 3'b001; ///
    localparam WRITE     = 3'b010;
    localparam READ_ADD  = 3'b011;
    localparam READ_DATA = 3'b100;

    input          MOSI, clk, rst_n, SS_n, tx_valid;
    input  [7:0] tx_data;
    output reg [9:0] rx_data;
    output reg      rx_valid, MISO;

    reg [3:0] counter;
    reg      received_address;

    reg [2:0] cs, ns;

    always @(posedge clk) begin
        if (~rst_n) begin
            cs <= IDLE;
        end
        else begin
            cs <= ns;
        end
    end
end

```



```

always @(*) begin
    case (cs)
        IDLE : begin
            if (SS_n)
                ns = IDLE;
            else
                ns = CHK_CMD;
        end
        CHK_CMD : begin
            if (SS_n)
                ns = IDLE;
            else begin
                if (~MOSI)
                    ns = WRITE;
                else begin
                    if (!received_address) // not(!) is added
                        ns = READ_ADD;
                    else
                        ns = READ_DATA;
                end
            end
        end
        WRITE : begin
            if (SS_n)
                ns = IDLE;
            else
                ns = WRITE;
        end
        READ_ADD : begin
            if (SS_n)
                ns = IDLE;
            else
                ns = READ_ADD;
        end
        READ_DATA : begin
            if (SS_n)
                ns = IDLE;
            else
                ns = READ_DATA;
        end
    endcase
end

```

```

always @(posedge clk) begin
    if (~rst_n) begin
        rx_data <= 0;
        rx_valid <= 0;
        counter <= 0; //
        received_address <= 0;
        MISO <= 0;
    end
    else begin
        case (cs)
            IDLE : begin
                rx_valid <= 0 ;
            end
            CHK_CMD : begin
                counter <= 10;
            end
            WRITE : begin
                if (counter > 0) begin
                    rx_data[counter-1] <= MOSI;
                    counter <= counter - 1;
                end
                else begin
                    rx_valid <= 1;
                end
            end
            READ_ADD : begin
                if (counter > 0) begin
                    rx_data[counter-1] <= MOSI;
                    counter <= counter - 1;
                end
                else begin
                    rx_valid <= 1;
                    received_address <= 1;
                end
            end
            READ_DATA : begin
                if (tx_valid) begin
                    rx_valid <= 0;
                    if (counter > 0) begin
                        MISO <= tx_data[counter-1];
                        counter <= counter - 1;
                    end
                    else begin
                        received_address <= 0;
                    end
                end
                else begin
                    if (counter > 0) begin
                        rx_data[counter-1] <= MOSI;
                        counter <= counter - 1;
                    end
                    else begin
                        rx_valid <= 1;
                        counter <= 9 ; //
                    end
                end
            end
        endcase
    end
end
end

```

```

`ifndef SIM
assert property (@(posedge clk) disable iff(!rst_n) (ISS_n && cs==IDLE) |-> ##1 cs==CHK_CMD);
assert property (@(posedge clk) disable iff(!rst_n) (ISS_n && cs==CHK_CMD && MOSI==0 ) |-> ##1 cs==WRITE);
assert property (@(posedge clk) disable iff(!rst_n) (ISS_n && cs==CHK_CMD && MOSI==1 ) |-> ##1 (cs==READ_ADD || cs==READ_DATA));
assert property (@(posedge clk) disable iff(!rst_n) (ISS_n && cs==CHK_CMD) |-> ##1 (cs==WRITE || cs==READ_ADD || cs==READ_DATA ));

assert property (@(posedge clk) disable iff(!rst_n) (SS_n && cs==READ_ADD && $past(cs)==CHK_CMD) |-> ##1 cs==IDLE);
assert property (@(posedge clk) disable iff(!rst_n) (SS_n && cs==READ_DATA && $past(cs)==CHK_CMD) |-> ##1 cs==IDLE);

cover property (@(posedge clk) disable iff(!rst_n) (ISS_n && cs==IDLE) |-> ##1 cs==CHK_CMD);
cover property (@(posedge clk) disable iff(!rst_n) (ISS_n && cs==CHK_CMD && MOSI==0 ) |-> ##1 cs==WRITE);
cover property (@(posedge clk) disable iff(!rst_n) (ISS_n && cs==CHK_CMD && MOSI==1 ) |-> ##1 (cs==READ_ADD || cs==READ_DATA));
cover property (@(posedge clk) disable iff(!rst_n) cs==CHK_CMD |-> ##1 (cs==WRITE || cs==READ_ADD || cs==READ_DATA ));
cover property (@(posedge clk) disable iff(!rst_n) cs==READ_ADD && $past(cs)==CHK_CMD |-> ##12 cs==IDLE);
cover property (@(posedge clk) disable iff(!rst_n) cs==READ_DATA && $past(cs)==CHK_CMD |-> ##22 cs==IDLE);

`endif

endmodule

```

3. SPI_slave golden design:

```
module SLAVE_golden_model (clk , SS_n , rst_n , MOSI , tx_data , tx_valid , MISO , rx_data , rx_valid );

parameter IDLE      = 3'b000 ;
parameter CHK_CMD   = 3'b001 ;
parameter WRITE     = 3'b010 ;
parameter READ_ADD  = 3'b011 ;
parameter READ_DATA = 3'b100 ;

input clk , SS_n , rst_n , MOSI ;
input [7:0] tx_data ;
input tx_valid ;

output reg MISO ;
output reg [9:0] rx_data ;
output reg rx_valid ;

(* fsm_encoding = "gray" *)
reg [2:0] cs , ns ;
reg read_data_or_address ;
reg [3:0] counter ;
```

```
// next state logic
always @(*) begin
    case (cs)
        // case IDLE
        IDLE :
            if(SS_n)
                ns = IDLE ;
            else
                ns = CHK_CMD ;

        // case CHK_CMD
        CHK_CMD :
            if(SS_n)
                ns = IDLE ;
            else if ( MOSI == 0 )
                ns = WRITE ;
            else if (~read_data_or_address)
                ns = READ_ADD ;
            else
                ns = READ_DATA ;

        // case WRITE
        WRITE :
            if(SS_n)
                ns = IDLE ;
            else
                ns = WRITE ;

        // case READ_ADD
        READ_ADD :
            if(SS_n)
                ns = IDLE ;
            else
                ns = READ_ADD ;

        // case READ_DATA
        READ_DATA :
            if(SS_n)
                ns = IDLE ;
            else
                ns = READ_DATA ;

    endcase
end
```

```

// state memory
always @(posedge clk) begin
    if(~rst_n)
        cs <= IDLE ;
    else
        cs <= ns ;
end

```

```

// output logic
always @(posedge clk) begin
    if(~rst_n) begin
        MISO          <= 0 ;
        rx_data        <= 10'b0 ;
        rx_valid        <= 0 ;
        read_data_or_address <= 0 ;
        counter         <= 0 ;
    end
    else begin
        case(cs)
            IDLE :
                rx_valid <= 0 ;
            CHK_CMD :
                counter <= 10 ;
            WRITE :
                if (counter > 0) begin
                    rx_data[counter-1] <= MOSI ;
                    counter <= counter - 1 ;
                end
                else begin
                    rx_valid <= 1 ;
                end
            READ_ADD :
                if (counter > 0) begin
                    rx_data[counter-1] <= MOSI ;
                    counter <= counter - 1 ;
                end
                else begin
                    rx_valid <= 1 ;
                    read_data_or_address <= 1 ;
                end
            READ_DATA :
                if (tx_valid) begin
                    rx_valid <= 0 ;
                    if(counter == 0) begin
                        read_data_or_address <= 0 ;
                    end
                    else begin
                        MISO <= tx_data[counter-1];
                        counter <= counter - 1 ;
                    end
                end
                else begin
                    /// dummy ;
                    if (counter > 0) begin
                        rx_data[counter-1] <= MOSI ;
                        counter <= counter - 1 ;
                    end
                    else begin
                        rx_valid <= 1 ;
                        counter <= 9 ;
                    end
                end
        endcase
    end
end
endmodule

```

4. SPI_slave interface:

```
interface SPI_if (clk);

    input clk ;
    logic MOSI ;
    logic rst_n ;
    logic SS_n ;
    logic [7:0] tx_data;
    logic tx_valid;

    // output for original design
    logic [9:0] rx_data;
    logic rx_valid;
    logic MISO;

    // output for golden design
    logic [9:0] rx_data_golden;
    logic rx_valid_golden;
    logic MISO_golden;

endinterface
```

5. SPI_slave config:

```
package SPI_config_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"

class SPI_config extends uvm_object ;

    `uvm_object_utils(SPI_config)

    virtual SPI_if SPI_vif;
    uvm_active_passive_enum is_active;

    function new(string name = "SPI_config");
        super.new(name);
    endfunction

endclass

endpackage
```

6. SPI_slave seq item:

```
package SPI_seq_item_pkg ;

import uvm_pkg::*;
`include "uvm_macros.svh"

typedef enum bit [2:0] {WRITE_ADDR = 3'b000, WRITE_DATA = 3'b001, READ_ADDR = 3'b110, READ_DATA = 3'b111} e_state;

class SPI_seq_item extends uvm_sequence_item;
    `uvm_object_utils(SPI_seq_item)

    rand bit rst_n      ;

    rand bit MOSI        ;
    rand bit SS_n        ;
    rand bit tx_valid     ;
    rand logic [7:0] tx_data ;

    logic [9:0] rx_data ;
    bit rx_valid        ;
    bit MISO            ;

    logic [9:0] rx_data_golden ;
    bit rx_valid_golden ;
    bit MISO_golden ;

    rand e_state ns;
    | e_state cs;

    int counter; // Counter for Serialization
    int SS_n_cnt; // Counter for Communication
    int WIDTH; // WIDTH = 13 (cs != READ_DATA) / 23 (cs = READ_DATA)

    rand bit MOSI_BUS_RAND[];

    bit MOSI_BUS[]; // [0:1:2]

    function new(string name = "SPI_seq_item");
        super.new(name);
    endfunction
```

```
// Responsible for Serialization Counter (counter) & cs reloading
function void pre_randomize ();
    if (!rst_n) begin
        counter = 0;
    end
    else begin
        if (SS_n_cnt == 0) begin // OFF COMM
            counter = 0;
            cs = ns;

            if (cs != READ_DATA) begin
                WIDTH = 13;
                MOSI_BUS = new[WIDTH];
                MOSI_BUS_RAND = new[WIDTH];
            end
            else begin
                WIDTH = 23;
                MOSI_BUS = new[WIDTH];
                MOSI_BUS_RAND = new[WIDTH];
            end
        end
        else begin
            // cs constant
            counter = counter + 1;
        end
    end
    // $display("rstn = %0b, MOSI = %0b, SS_n = %0b, SS_n_cnt = %0d, counter = %0d, MOSI_BUS = %0p, cs = %0p, ns = %0p, tx_vali
endfunction
```

```

// Responsible for Communication Counter (ss_n_cnt)
function void post_randomize ();
    if (!rst_n) begin
        SS_n_cnt = 0;
    end
    else begin
        MOSI_BUS = MOSI_BUS_RAND;
        for (int i = 0 ; i < 3 ; i++) begin
            MOSI_BUS [i] = cs[2-i];           // MOSI_BUS[0:2] = cs;
        end

        if ((cs != READ_DATA) && (SS_n_cnt != 13))
            SS_n_cnt += 1;
        else if ((cs == READ_DATA) && (SS_n_cnt != 23))
            SS_n_cnt += 1;
        else
            SS_n_cnt = 0;
    end
endfunction

```

```

constraint rst_n_c { rst_n dist { 0:/2 , 1:/98 }; }

constraint state_c {
    if (!rst_n) {
        ns == e_state'(0);
    }
    else {
        ns inside {WRITE_ADDR,WRITE_DATA,READ_ADDR,READ_DATA};
        if (cs == WRITE_ADDR)
            ns inside {WRITE_ADDR, WRITE_DATA};
        else if (cs == WRITE_DATA)
            ns dist {WRITE_ADDR :/30, READ_ADDR :/70};
        else if (cs == READ_ADDR)
            ns == READ_DATA;
        else if (cs == READ_DATA)
            ns dist {WRITE_ADDR :/30, READ_ADDR :/70};
    }
}

constraint mosi_c {
    if (!rst_n) {
        MOSI == 0;
    }
    else {
        if (SS_n_cnt != WIDTH)           // MSB      LSB
            MOSI == MOSI_BUS[counter]; // [0:1:2...WIDTH]
    }
}

```

```

constraint ss_n_c {
    if (!rst_n) {
        SS_n == 1;
    }
    else {
        if (cs != READ_DATA) {
            if (SS_n_cnt < 13)
                SS_n == 0;
            else
                SS_n == 1;
        }
        else {
            if (SS_n_cnt < 23)
                SS_n == 0;
            else
                SS_n == 1;
        }
    }
}

constraint tx_valid_c {
    if (!rst_n) {
        tx_valid == 0;
    }
    else {
        if (!SS_n) {
            if ((cs == READ_DATA) && SS_n_cnt > 12)
                tx_valid == 1;
            else
                tx_valid == 0;
        }
    }
}

endclass
endpackage

```

7. SPI_slave main seq:

```

package SPI_main_seq_pkg;
import uvm_pkg::*;
import SPI_seq_item_pkg::*;
`include "uvm_macros.svh"

class SPI_main_seq extends uvm_sequence #(SPI_seq_item);
    `uvm_object_utils(SPI_main_seq);

    SPI_seq_item seq_item ;

    function new(string name = "SPI_main_seq");
        super.new(name);
    endfunction

    task body;
        seq_item = SPI_seq_item::type_id::create("seq_item");

        repeat(10000)begin
            start_item(seq_item);
            assert(seq_item.randomize());
            finish_item(seq_item);
        end
    endtask
endclass
endpackage

```


8. SPI_slave reset seq:

```
package SPI_reset_seq_pkg;
import uvm_pkg::*;
import SPI_seq_item_pkg::*;
`include "uvm_macros.svh"

class SPI_reset_seq extends uvm_sequence #(SPI_seq_item);
    `uvm_object_utils(SPI_reset_seq)

    SPI_seq_item seq_item;

    function new(string name = "SPI_reset_seq");
        super.new(name);
    endfunction

    task body;
        seq_item = SPI_seq_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n = 0 ;

        seq_item.MOSI = 0 ;
        seq_item.SS_n = 1 ;
        seq_item.tx_valid = 0 ;
        seq_item.tx_data = 0 ;
        finish_item(seq_item);
    endtask
endclass
endpackage
```

9. SPI_slave driver:

```
package SPI_driver_pkg;
import uvm_pkg::*;
import SPI_seq_item_pkg::*;
`include "uvm_macros.svh"

class SPI_driver extends uvm_driver #(SPI_seq_item);
    `uvm_component_utils(SPI_driver)

    virtual SPI_if SPI_vif ;
    SPI_seq_item stim_seq_item ;

    function new(string name = "SPI_driver" , uvm_component parent = null);
        super.new(name, parent);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            stim_seq_item = SPI_seq_item::type_id::create("stim_seq_item");
            seq_item_port.get_next_item(stim_seq_item);
            SPI_vif.MOSI = stim_seq_item.MOSI ;
            SPI_vif.rst_n = stim_seq_item.rst_n ;
            SPI_vif.SS_n = stim_seq_item.SS_n ;
            SPI_vif.tx_valid = stim_seq_item.tx_valid ;
            SPI_vif.tx_data = stim_seq_item.tx_data ;
            @(negedge SPI_vif.clk);

            seq_item_port.item_done();
        end
    endtask
endclass
endpackage
```

10. SPI_slave monitor:

```
package SPI_monitor_pkg;
import uvm_pkg::*;
import SPI_seq_item_pkg::*;
`include "uvm_macros.svh"

class SPI_monitor extends uvm_monitor;
    `uvm_component_utils(SPI_monitor)

    virtual SPI_if SPI_vif;
    SPI_seq_item rsp_seq_item;
    uvm_analysis_port #(SPI_seq_item) mon_ap;

    function new(string name = "SPI_monitor" , uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        mon_ap = new("mon_ap", this);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            rsp_seq_item = SPI_seq_item::type_id::create("rsp_seq_item");
            @(negedge SPI_vif.clk);

            rsp_seq_item.MOSI      = SPI_vif.MOSI      ;
            rsp_seq_item.rst_n     = SPI_vif.rst_n     ;
            rsp_seq_item.SS_n      = SPI_vif.SS_n      ;
            rsp_seq_item.tx_valid  = SPI_vif.tx_valid  ;
            rsp_seq_item.tx_data   = SPI_vif.tx_data   ;

            rsp_seq_item.rx_data   = SPI_vif.rx_data   ;
            rsp_seq_item.rx_valid  = SPI_vif.rx_valid  ;
            rsp_seq_item.MISO      = SPI_vif.MISO      ;

            rsp_seq_item.rx_data_golden = SPI_vif.rx_data_golden ;
            rsp_seq_item.rx_valid_golden = SPI_vif.rx_valid_golden ;
            rsp_seq_item.MISO_golden   = SPI_vif.MISO_golden   ;

            mon_ap.write(rsp_seq_item);
        end
    endtask
endclass
endpackage
```

11. SPI_slave_sequencer:

```
package SPI_sequencer_pkg;
import uvm_pkg::*;
import SPI_seq_item_pkg::*;
`include "uvm_macros.svh"

class SPI_sequencer extends uvm_sequencer #(SPI_seq_item);
    `uvm_component_utils(SPI_sequencer);

    function new(string name = "SPI_sequencer" , uvm_component parent = null);
        super.new(name,parent);
    endfunction
endclass
endpackage
```

12. SPI_slave_agent:

```
package SPI_agent_pkg;
import uvm_pkg::*;
import SPI_sequencer_pkg::*;
import SPI_driver_pkg::*;
import SPI_monitor_pkg::*;
import SPI_config_pkg::*;
import SPI_seq_item_pkg::*;
`include "uvm_macros.svh"

class SPI_agent extends uvm_agent;
    `uvm_component_utils(SPI_agent)

    SPI_sequencer sqr ;
    SPI_driver drv ;
    SPI_monitor mon ;
    SPI_config SPI_cfg;
    uvm_analysis_port #(SPI_seq_item) agt_ap;

    function new(string name = "SPI_agent" , uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db #(SPI_config)::get(this , "" , "SLAVE_CFG" , SPI_cfg ))begin
            `uvm_fatal("build_phase", "unable to get configuration object")
        end
        if(SPI_cfg.is_active == UVM_ACTIVE)begin
            sqr = SPI_sequencer::type_id::create("sqr",this);
            drv = SPI_driver::type_id::create("drv",this);
        end
        mon = SPI_monitor::type_id::create("mon",this);
        agt_ap = new("agt_ap",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        if(SPI_cfg.is_active == UVM_ACTIVE)begin
            drv.SPI_vif = SPI_cfg.SPI_vif ;
            drv.seq_item_port.connect(sqr.seq_item_export);
        end
        mon.SPI_vif = SPI_cfg.SPI_vif ;
        mon.mon_ap.connect(agt_ap);
    endfunction
endclass
endpackage
```

13. SPI_slave_coverage

```
package SPI_coverage_pkg;
import uvm_pkg::*;
import SPI_seq_item_pkg::*;
`include "uvm_macros.svh"

class SPI_coverage extends uvm_component;
    `uvm_component_utils(SPI_coverage)

    uvm_analysis_export #(SPI_seq_item) cov_export ;
    uvm_tlm_analysis_fifo #(SPI_seq_item) cov_fifo;
    SPI_seq_item seq_item_cov;

    /// covergroups
    covergroup SPI_cg;
        cp_rx: coverpoint seq_item_cov.rx_data[9:8] {
            bins all_values[] = {[0:3]};
            bins WRITE_ADD    = (2'b00=>2'b00=>2'b00);
            bins WRITE_DATA   = (2'b00=>2'b00=>2'b01);
            bins READ_ADD     = (2'b00=>2'b10=>2'b10);
            bins READ_DATA    = (2'b00=>2'b10=>2'b11);
        }
        cp_SS_n: coverpoint seq_item_cov.SS_n {
            bins normal_txn = (1 => 0 [*13] => 1);
            bins extended_txn = (1 => 0 [*23] => 1);
            bins start_comm = (1=>0);
            bins comm = {0};
        }
        cp_MOSI: coverpoint seq_item_cov.MOSI{
            bins write_addr = (0=>0=>0);
            bins write_data = (0=>0=>1);
            bins read_addr  = (1=>1=>0);
            bins read_data  = (1=>1=>1);
        }

        // cross
        cross cp_SS_n, cp_MOSI {
            ignore_bins invalid0 = binsof(cp_SS_n.extended_txn)
            |&& binsof(cp_MOSI.read_data) ;

            ignore_bins invalid1 = binsof(cp_SS_n.normal_txn)
            |&& binsof(cp_MOSI.read_data);

            ignore_bins invalid2 = binsof(cp_SS_n.extended_txn)
            |&& binsof(cp_MOSI.write_data);
        }
    endgroup
endgroup
```

14. SPI_slave_scoreboard:

```
package SPI_scoreboard_pkg;
import uvm_pkg::*;
import SPI_seq_item_pkg::*;
`include "uvm_macros.svh"

class SPI_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(SPI_scoreboard)

    uvm_analysis_export #(SPI_seq_item) sb_export;
    uvm_tlm_analysis_fifo #(SPI_seq_item) sb_fifo;

    SPI_seq_item seq_item_sb;

    int error_count    = 0 ;
    int correct_count = 0 ;

    function new(string name = "SPI_scoreboard", uvm_component parent = null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sb_export = new("sb_export", this);
        sb_fifo = new("sb_fifo", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        sb_export.connect(sb_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            sb_fifo.get(seq_item_sb);
            if( (seq_item_sb.rx_data_golden != seq_item_sb.rx_data) ||
                (seq_item_sb.rx_valid != seq_item_sb.rx_valid_golden) ||
                (seq_item_sb.MISO != seq_item_sb.MISO_golden) ) begin
                error_count++;
                `uvm_error("run_phase", $sformatf("error"));
            end
            else begin
                correct_count++;
            end
        end
    endtask

    function void report_phase(uvm_phase phase);
        super.report_phase(phase);
        `uvm_info("report_phase",$sformatf("Total Successful Transactions: %0d",correct_count),UVM_MEDIUM);
        `uvm_info("report_phase",$sformatf("Total Failed Transactions: %0d",error_count),UVM_MEDIUM);
    endfunction

endclass
endpackage
```

15. SPI_Slave env:

```
package SPI_env_pkg;
import uvm_pkg::*;
import SPI_agent_pkg::*;
import SPI_scoreboard_pkg::*;
import SPI_coverage_pkg::*;
`include "uvm_macros.svh"

class SPI_env extends uvm_env;
    `uvm_component_utils(SPI_env)

    SPI_agent agt;
    SPI_scoreboard sb ;
    SPI_coverage cov ;

    function new(string name = "SPI_env", uvm_component parent = null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agt = SPI_agent::type_id::create("agt",this);
        sb = SPI_scoreboard::type_id::create("sb",this);
        cov = SPI_coverage::type_id::create("cov",this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agt.agt_ap.connect(sb.sb_export);
        agt.agt_ap.connect(cov.cov_export);
    endfunction
endclass
endpackage
```

16. SPI_slave_test:

```
package SPI_test_pkg;

import uvm_pkg::*;
import SPI_env_pkg::*;
import SPI_config_pkg::*;
import SPI_main_seq_pkg::*;
import SPI_reset_seq_pkg::*;
`include "uvm_macros.svh"

class SPI_test extends uvm_test;
    `uvm_component_utils(SPI_test)

    SPI_env env;
    SPI_config SPI_cfg;
    virtual SPI_if SPI_vif;
    SPI_main_seq main_seq;
    SPI_reset_seq reset_seq;

    function new(string name = "SPI_test" , uvm_component parent = null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = SPI_env::type_id::create("env",this);
        SPI_cfg = SPI_config::type_id::create("SPI_cfg");
        main_seq = SPI_main_seq::type_id::create("main_seq");
        reset_seq = SPI_reset_seq::type_id::create("reset_seq");

        if(!uvm_config_db #(virtual SPI_if)::get( this , "" , "SPI_IF" , SPI_cfg.SPI_vif ))
            `uvm_fatal("build_phase", "test-unable to get the virtual interface");

        SPI_cfg.is_active = UVM_ACTIVE;
        uvm_config_db #(SPI_config)::set(this , "" , "SLAVE_CFG" , SPI_cfg);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        phase.raise_objection(this);

        reset_seq.start(env.agt.sqr);
        main_seq.start(env.agt.sqr);

        phase.drop_objection(this);
    endtask

endclass
endpackage
```

17. SPI_slave top:

```
import uvm_pkg::*;
`include "uvm_macros.svh"

import SPI_test_pkg::*;

module top();

    bit clk ;

    initial begin
        forever
            #1 clk = ~clk ;
    end

    SPI_if SPIif(clk);

    SLAVE DUT (SPIif.MOSI, SPIif.MISO, SPIif.SS_n, SPIif.clk, SPIif.rst_n,
    | | | | | SPIif.rx_data, SPIif.rx_valid, SPIif.tx_data, SPIif.tx_valid);

    SLAVE_golden_model golden (SPIif.clk, SPIif.SS_n, SPIif.rst_n, SPIif.MOSI, SPIif.tx_data, SPIif.tx_valid,
    | | | | | SPIif.MISO_golden, SPIif.rx_data_golden, SPIif.rx_valid_golden );

    bind DUT SLAVE_sva slave_bind (SPIif.MOSI, SPIif.MISO, SPIif.SS_n, SPIif.clk, SPIif.rst_n,
    | | | | | SPIif.rx_data, SPIif.rx_valid, SPIif.tx_data, SPIif.tx_valid);

    initial begin
        uvm_config_db #(virtual SPI_if)::set (null, "uvm_test_top", "SPI_IF", SPIif );
        run_test("SPI_test");
    end

endmodule
```

Activate

Running all files in questa:

```
# Compile of SPI_slave_golden_model.sv was successful.
# Compile of SPI_slave.sv was successful.
# Compile of SPI_sva.sv was successful.
# Compile of SPI_if.sv was successful.
# Compile of 02-SPI_seq_item.sv was successful.
# Compile of 03-SPI_main_seq.sv was successful.
# Compile of 04-SPI_reset_seq.sv was successful.
# Compile of 09-SPI_coverage.sv was successful.
# Compile of 12-SPI_test.sv was successful.
# Compile of 13-SPI_top.sv was successful.
# Compile of 08-SPI_agent.sv was successful.
# Compile of 01-SPI_config.sv was successful.
# Compile of 10-SPI_scoreboard.sv was successful.
# Compile of 11-SPI_env.sv was successful.
# Compile of 07-SPI_sequencer.sv was successful.
# Compile of 06-SPI_monitor.sv was successful.
# Compile of 05-SPI_driver.sv was successful.
QuestaSim> files, 0 failed with no errors.
```


18. Code Coverage:

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Me
/SPI_coverage_pkg/SPI_coverage		100.00%					
TYPE SPI_cg		100.00%	100	100.00...			
CVP SPI_cg::cp_rx		100.00%	100	100.00...			
CVP SPI_cg::cp_SS_n		100.00%	100	100.00...			
CVP SPI_cg::cp_MOSI		100.00%	100	100.00...			
CROSS SPI_cg::(#cross_0#)		100.00%	100	100.00...			
INST \SPI_coverage_pkg::SPI_coverage::SPI_cg		100.00%	100	100.00...			
CVP cp_rx		100.00%	100	100.00...			
bin all_values[0]		2791	1	100.00...			
bin all_values[1]		1966	1	100.00...			
bin all_values[2]		2568	1	100.00...			
bin all_values[3]		2676	1	100.00...			
bin WRITE_ADD		2220	1	100.00...			
bin WRITE_DATA		58	1	100.00...			
bin READ_ADD		94	1	100.00...			
bin READ_DATA		105	1	100.00...			
CVP cp_SS_n		100.00%	100	100.00...			
bin normal_bn		352	1	100.00...			
bin extended_bn		146	1	100.00...			
bin start_comm		671	1	100.00...			
bin comm		9320	1	100.00...			
CVP cp_MOSI		100.00%	100	100.00...			
bin write_addr		1501	1	100.00...			
bin write_data		1288	1	100.00...			
bin read_addr		1214	1	100.00...			
bin read_data		1046	1	100.00...			
CROSS #cross_0#		100.00%	100	100.00...			
bin <comm_read_data>		985	1	100.00...			
bin <comm_read_addr>		1103	1	100.00...			
bin <extended_bn_read_addr>		21	1	100.00...			
bin <start_comm_read_addr>		127	1	100.00...			
bin <normal_bn_read_addr>		47	1	100.00...			
bin <comm_write_data>		1223	1	100.00...			
bin <start_comm_write_data>		58	1	100.00...			
bin <normal_bn_write_data>		50	1	100.00...			
bin <comm_write_addr>		1395	1	100.00...			
bin <extended_bn_write_addr>		17	1	100.00...			
bin <start_comm_write_addr>		154	1	100.00...			
bin <normal_bn_write_addr>		43	1	100.00...			
ignore_bin invalid3		0	-	-			
ignore_bin invalid2		15	-	-			
ignore_bin invalid1		41	-	-			

19. Assertions:

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
/top/DUT/cover__5	SVA	Off	Off	92	1	Unk...	1	100%			0	0	0 ns	0
/top/DUT/cover__4	SVA	Off	Off	109	1	Unk...	1	100%			0	0	0 ns	0
/top/DUT/cover__3	SVA	Off	Off	646	1	Unk...	1	100%			0	0	0 ns	0
/top/DUT/cover__2	SVA	Off	Off	398	1	Unk...	1	100%			0	0	0 ns	0
/top/DUT/cover__1	SVA	Off	Off	248	1	Unk...	1	100%			0	0	0 ns	0
/top/DUT/cover__0	SVA	Off	Off	656	1	Unk...	1	100%			0	0	0 ns	0
/top/DUT/plave_band/cover__3	SVA	Off	Off	59	1	Unk...	1	100%			0	0	0 ns	0
/top/DUT/plave_band/cover__2	SVA	Off	Off	59	1	Unk...	1	100%			0	0	0 ns	0
/top/DUT/plave_band/cover__1	SVA	Off	Off	57	1	Unk...	1	100%			0	0	0 ns	0
/top/DUT/plave_band/cover__0	SVA	Off	Off	66	1	Unk...	1	100%			0	0	0 ns	0
/top/DUT/plave_band/cover__p_reset_outputs_low	SVA	Off	Off	200	1	Unk...	1	100%			0	0	0 ns	0

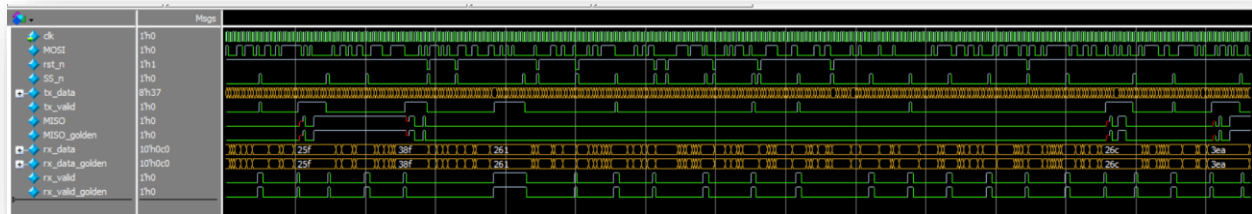
Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion Expression	Included
inmed__1735	Immediate	SVA	on	0	0	-	-	-	-	0	off	assert (\$cast(seq,o))	✗
inmed__1775	Immediate	SVA	on	0	0	-	-	-	-	0	off	assert (\$cast(seq,o))	✗
inmed__20	Immediate	SVA	on	0	1	-	-	-	-	0	off	assert (randomize(...))	✗
assert__0	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...)	✓
assert__1	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...)	✓
assert__2	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...)	✓
assert__3	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...)	✓
assert__4	Concurrent	SVA	on	0	0	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...)	✓
assert__5	Concurrent	SVA	on	0	0	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...)	✓
assert__p_reset_outputs_low	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) (~rst_n))>...	✓
assert__0	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...)	✓
assert__1	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...)	✓
assert__2	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...)	✓
assert__3	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) disable iff (...)	✓

```

# *****
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 20002: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO 10-SPI_scoreboard.sv(49) @ 20002: uvm_test_top.env.sb [report_phase] Total Successful Transactions: 10001
# UVM_INFO 10-SPI_scoreboard.sv(50) @ 20002: uvm_test_top.env.sb [report_phase] Total Failed Transactions: 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 6
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
#
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [report_phase] 2
#
# ** Note: $finish : C:/questasim64_2021.1/win64/.../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 20002 ns Iteration: 61 Instance: /top
#
# 1
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2021.1/win64/.../verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
#
/SIM 2>

```

20. Waveform:



21. Assertions Table

Feature	Assertion
At the first positive clock edge after reset is deasserted, the MISO line, rx_valid signal, and rx_data bus must all be low.	@(posedge clk) (!rst_n) -> ##1 (!MISO && !rx_valid && ~rx_data);
Three clock cycles after the chip select becomes active, the MOSI line must be low for three consecutive cycles	(\$fell(SS_n) ##1 !MOSI ##1 !MOSI ##1 !MOSI);
Three clock cycles after the chip select becomes active, the MOSI line is low for two cycles and then high on the third	(\$fell(SS_n) ##1 !MOSI ##1 !MOSI ##1 MOSI);
Three clock cycles after the chip select becomes active, the MOSI line is high for two cycles and then low on the third	(\$fell(SS_n) ##1 MOSI ##1 MOSI ##1 !MOSI);
Three clock cycles after the chip select becomes active, the MOSI line must be high for three consecutive cycles	(\$fell(SS_n) ##1 MOSI ##1 MOSI ##1 MOSI);

22. Bug report

- 1) The condition should check for not received_address to remain in the READ_ADDR , else should Transition to the READ_DATA.
- 2) The design should should reload the counter to 9 as there will be a waiting cycle before the tx_valid comes and the Data gets out starting from address 7 in the tx_data.
- 3) localparameter CHK_CMD = 3'b001;

b. Part 2: UVM Environment for Single-Port RAM

1. Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functional Checking
RAM_1	if rst_n =0 dout,tx_valid,Rd_Addr & Wr_Addr will be equal to zero	rst is made one time first by task then randomized under constraint to be deasserted by 95% and asserted 5%	NA	Output Checked against dout_ref (reference model) compared to that in the RAM and also by assertion
RAM_2	if din [9:8] din =11 then should be tx_valid =1 only in this case	No Randomization for tx_valid as it is output	Covered by cross coverage between 2 coverpoint Transaction_ordering_cp & tx_valid_high_cp	Output Checked against dout_ref (reference model) compared to that in the RAM and also by assertion
RAM_3	rx_valid should be equal =1 to read & write	Randomized under constraint to be asserted 95% and deasserted 5%	Covered by cross coverage between 2 coverpoint Transaction_ordering_cp & rx_valid_high_cp	Output Checked against dout_ref (reference model) compared to that in the RAM and also by assertion
RAM_4	din	Randomization under constraints "read_write_order_c" on MOSI where every Write Address is followed by either Write Address or Write Data, every Read Address is followed by Read Data, and post-data operations follow weighted probabilities: after Write Data → 60% Read Address / 40% Write Address, after Read Data → 60% Write Address / 40% Read Address	covered by coverpoint Transaction_ordering_cp	Output Checked against dout_ref (reference model) compared to that in the RAM and also by assertion

2. Design of RAM without Bugs:

```

module RAM (input bit clk, bit [9:0] din, bit rst_n, bit
rx_valid, output bit tx_valid, logic [7:0] dout);

reg [7:0] MEM [255:0];
reg [7:0] Rd_Addr, Wr_Addr;

always @(posedge clk) begin
    if (!rst_n) begin
        dout <= 0;
        tx_valid <= 0;
        Rd_Addr <= 0;
        Wr_Addr <= 0;
    end
    else begin //put begin &
end
        if (rx_valid) begin
            case (din[9:8])
                2'b00 : begin
                    Wr_Addr <= din[7:0];
                    tx_valid <= 0;
                end
                2'b01 : begin
                    MEM[Wr_Addr] <= din[7:0];
                    tx_valid <= 0;
                end
                2'b10 : begin

```

```

        Rd_Addr <= din[7:0];
        tx_valid <= 0;
    end
    2'b11 : begin
        dout <= MEM[Rd_Addr]; //bug MEM[Rd_Addr] instead
of MEM[Wr_Addr]
        tx_valid <= 1;
    end
    default : tx_valid <= 0;
endcase
end
end
end
endmodule

```

3. Assertions of RAM:

```

module SVA(input bit clk, bit [9:0] din, bit rst_n, bit
rx_valid, bit tx_valid, logic [7:0] dout);

property p1;
@(posedge clk) (!rst_n) | => ( tx_valid==1'b0 &&
dout==8'b0); //Scheduling semantics
endproperty
Assert_reset: assert property(p1);
cover_reset: cover property(p1);

property p2;
@(posedge clk) disable iff(!rst_n) (din[9:8]==2'b00 ||
din[9:8]==2'b01 || din[9:8]==2'b10) | => (tx_valid==0);
endproperty
Assert_tx_valid: assert property(p2);
cover_tx_valid: cover property(p2);

property p3;

```

```

@(posedge clk) disable iff(!rst_n) (din[9:8]==2'b11) | =>
(tx_valid==1) | => (din[9:8]==2'b00||din[9:8]==2'b10) | =>
(tx_valid==0) ;
endproperty
Assert_tx_valid_fell: assert property(p3);
cover_tx_valid_fell:cover property(p3);

property p4;//we did [=1:$] because 2'b00 can come more than
one time consecutively
@(posedge clk) disable iff(!rst_n) (din[9:8]==2'b00) [=1:$]
##1 (din[9:8]==2'b01);
endproperty
Assert_wr_add_to_wr_data: assert property(p4);
cover_wr_add_to_wr_data:cover property(p4);

property p5;//we did [=1:$] because 2'b10 can come more than
one time consecutively
@(posedge clk) disable iff(!rst_n) (din[9:8]==2'b10) [=1:$]
##1 (din[9:8]==2'b11);
endproperty
Assert_rd_add_to_rd_data: assert property(p5);
cover_rd_add_to_rd_data:cover property(p5);

endmodule

```

4. Configuration object of RAM:

```
package pkg_config;
import uvm_pkg::*;
`include "uvm_macros.svh"
class ram_config_obj extends uvm_object;
`uvm_object_utils(ram_config_obj)
virtual ram_interface ram_vif;
uvm_active_passive_enum is_active;

function new(string name="ram_config_obj");
super.new(name);
endfunction
endclass
endpackage
```

5. Interface of RAM:

```
interface ram_interface (clk);
//INPUTS
input bit clk;
bit rst_n, rx_valid;
bit [9:0] din;
//OUTPUTS
bit tx_valid; //Should be high if we want to read
logic [7:0] dout;

modport DUT( input clk, din, rst_n, rx_valid, output
tx_valid, dout);
modport sva(input clk, din, rst_n,
rx_valid, tx_valid, dout);
endinterface
```

6. Top of RAM:

```
import pkg_test::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
module top();
bit clk,rst_n;
initial begin
    clk=0;
    forever #1 clk=~clk;
end
//instantiations
ram_interface intf(clk);
RAM DUT(intf);
SVA assertion(intf);
bind RAM SVA binding_all(intf);
initial begin
uvm_config_db #(virtual
ram_interface)::set(null,"uvm_test_top","ram_interface",intf);
$readmemb("mem.dat", DUT.MEM );//initializing memory
run_test("ram_test");
end
```

7. Test of RAM:

```
package pkg_test;
import agt_pkg::*;
import pkg_driver::*;
import uvm_pkg::*;
import seq_item_pkg::*;
import seq_pkg::*;
import sequencer_pkg::*;
import pkg_config::*;
import pkg_env::*;
`include "uvm_macros.svh"
class ram_test extends uvm_test;
`uvm_component_utils(ram_test)
```



```

virtual ram_interface ram_vif;
//build main sequence class & env class
ram_env env;
ram_config_obj ram_config;
reset_sequence reset_seq ;
write_only_sequence write_only_seq;
read_only_sequence read_only_seq;
write_read_sequence write_read_seq;
ram_agent agt;

//Provide the constructor default values
function new(string name="ram_test", uvm_component parent =null) ;
super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);

env=ram_env::type_id::create("env",this);
ram_config= ram_config_obj::type_id::create("ram_config");
reset_seq= reset_sequence ::type_id::create("reset_seq");
write_only_seq= write_only_sequence
::type_id::create("write_only_seq");
read_only_seq=
read_only_sequence::type_id::create("read_only_seq");
write_read_seq= write_read_sequence
::type_id::create("write_read_seq");

if(!uvm_config_db#(virtual ram_interface)::get
(this,"","ram_interface",ram_config.ram_vif))
`uvm_fatal("build_phase","Unable to get virtual interface");

uvm_config_db #(ram_config_obj)::set(this,"*","CFG",ram_config);

endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);
phase.raise_objection(this);

```

```

`uvm_info("run_phase","stimulus generated started",UVM_LOW);

reset_seq.start(env.agt.sqr);
write_only_seq.start(env.agt.sqr);
read_only_seq.start(env.agt.sqr);
write_read_seq.start(env.agt.sqr);

`uvm_info("run_phase","stimulus generated ended",UVM_LOW);
phase.drop_objection(this);
endtask
endclass
endpackage

```

8. Environment of RAM:

```

package pkg_env;
import agt_pkg::*;
import scoreboard_pkg::*;
import cov_pkg::*;
import pkg_driver::*;
import uvm_pkg::*;
import seq_item_pkg::*;
import seq_pkg::*;
import sequencer_pkg::*;
`include "uvm_macros.svh"

class ram_env extends uvm_env;
`uvm_component_utils(ram_env);

ram_agent agt;
ram_scoreboard sb;
ram_coverage cov;

function new(string name="ram_env",uvm_component parent =null)
;

```

```

super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);

agt=ram_agent::type_id::create("agt",this);
sb=ram_scoreboard::type_id::create("sb",this);
cov=ram_coverage::type_id::create("cov",this);
endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
agt.agt_ap.connect(sb.sb_export);
agt.agt_ap.connect (cov.cov_export);
endfunction
endclass
endpackage

```

9. Agent of RAM:

```

package agt_pkg;
`include "uvm_macros.svh"
import uvm_pkg::*;
import pkg_driver::*;
import mon_pkg::*;
import pkg_config::*;
import sequencer_pkg::*;
import seq_item_pkg::*;
import seq_pkg::*;

class ram_agent extends uvm_agent;
`uvm_component_utils(ram_agent);

ram_driver drv; //Create object for driver

```

```

ram_sequencer sqr;//Create object for sequencer
ram_monitor mon;//Create object for monitor
ram_config_obj ram_config;//Create object for config_object

uvm_analysis_port #(ram_seq_item) agt_ap;

function new(string name="ram_agent",uvm_component parent =null) ;
super.new(name,parent);
endfunction

//This function to Place it in the uvm hierarchy without this function we cannot use this class
//you cannot make object without this function
//super.new ==> to go to parent uvm_agent and extends from it and identify who will be parent to this agent

function void build_phase(uvm_phase phase);
super.build_phase(phase);

if(!uvm_config_db
#(ram_config_obj)::get(this,"","RAM_CFG",ram_config))
`uvm_fatal("build_phase","Unable to get configuration object of driver");
if(ram_config.is_active == UVM_ACTIVE)begin
    drv=ram_driver::type_id:: create("drv",this);
    sqr=ram_sequencer::type_id:: create("sqr",this);
end
mon=ram_monitor::type_id:: create("mon",this);
agt_ap=new("agt_ap",this);

endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
if(ram_config.is_active == UVM_ACTIVE)begin
    //driver
    drv.ram_vif=ram_config.ram_vif;
    drv.seq_item_port.connect(sqr.seq_item_export);//TLM
end

```

```
//monitor
mon.ram_vif=ram_config.ram_vif;
mon.mon_ap.connect(agt_ap);
endfunction
endclass
endpackage
```

10. Sequence of RAM:

```
package seq_pkg;
`include "uvm_macros.svh"
import uvm_pkg::*;
import seq_item_pkg::*;

class reset_sequence extends uvm_sequence #(ram_seq_item);
`uvm_object_utils(reset_sequence)

ram_seq_item seq_item;

function new(string name="reset_sequence");
super.new(name);
endfunction

//Reset
task body();
seq_item=ram_seq_item::type_id::create("seq_item");//put the
object created in factory
$display("START OF RESET");
start_item(seq_item);

seq_item.din=0;
seq_item.rx_valid=0;
seq_item.rst_n=0;
finish_item(seq_item);
#1 $display("END OF RESET");
```

```

endtask
endclass

//WRITE ONLY
class write_only_sequence extends uvm_sequence
#(ram_seq_item);
`uvm_object_utils(write_only_sequence)

ram_seq_item seq_item;

function new(string name="write_only_sequence");
super.new(name);
endfunction

task body();
seq_item=ram_seq_item::type_id::create("seq_item");//put the
object created in factory
seq_item.read_only.constraint_mode(0);
seq_item.write_only.constraint_mode(1);
seq_item.mix_read_write.constraint_mode(0);
$display("START OF WRITE ONLY");
repeat(10000) begin
start_item(seq_item);

assert(seq_item.randomize());

finish_item(seq_item);
end
$display("END OF WRITE ONLY");
endtask
endclass

```

```

//READ ONLY
class read_only_sequence extends uvm_sequence #(ram_seq_item);
`uvm_object_utils(read_only_sequence)

ram_seq_item seq_item;

function new(string name="read_only_sequence");
super.new(name);
endfunction

task body();
seq_item=ram_seq_item::type_id::create("seq_item");//put the
object created in factory
seq_item.read_only.constraint_mode(1);
seq_item.write_only.constraint_mode(0);
seq_item.mix_read_write.constraint_mode(0);
$display("START OF READ ONLY");
repeat(10000) begin
start_item(seq_item);

assert(seq_item.randomize());

finish_item(seq_item);
end
$display("END OF READ ONLY");
endtask
endclass

```

```

//WRITE & READ
class write_read_sequence extends uvm_sequence
#(ram_seq_item);
`uvm_object_utils(write_read_sequence)

ram_seq_item seq_item;

function new(string name="write_read_sequence");
super.new(name);
endfunction

task body();
seq_item=ram_seq_item::type_id::create("seq_item");//put the
object created in factory
seq_item.read_only.constraint_mode(0);
seq_item.write_only.constraint_mode(0);
seq_item.mix_read_write.constraint_mode(1);
$display("START OF WRITE & READ");
repeat(10000) begin

start_item(seq_item);

assert(seq_item.randomize());

finish_item(seq_item);
end
$display("END OF WRITE & READ");
Endtask
endclass
endpackage

```


11. Sequence item of RAM:

```
package seq_item_pkg;
`include "uvm_macros.svh"
import uvm_pkg::*;

class ram_seq_item extends uvm_sequence_item;
`uvm_object_utils(ram_seq_item)
rand bit rst_n,rx_valid;
rand bit [9:0] din;
bit tx_valid;
logic [7:0] dout;
bit [1:0] prev_din;

function new(string name="ram_seq_item");
super.new(name);
endfunction

function string convert2string();
return $sformatf("%s din=0b%b, rx_valid=0b%b ,rst_n= 0b%b,
tx_valid= 0b%b ,dout= 0b%b",
super.convert2string(),din ,rx_valid ,rst_n , tx_valid , dout);
endfunction

function string convert2string_stimulus();
return $sformatf("%s din=0b%b, rx_valid=0b%b ,rst_n= 0b%b,
tx_valid= 0b%b ,dout= 0b%b",
super.convert2string(),din ,rx_valid ,rst_n , tx_valid , dout);
endfunction

//CONSTRAINTS

constraint ram{
    rst_n dist{1:/95, 0:/5};
    rx_valid dist{1:/95, 0:/5};
}

constraint write_only{
```

```

    (prev_din==2'b00)->{din[9:8] dist{2'b00:/30, 2'b01:/70}};
    (prev_din==2'b01)->{din[9:8] dist{2'b01:/30, 2'b00:/70}};
}

constraint read_only{
    (prev_din==2'b10)->{din[9:8]==2'b11};
    (prev_din==2'b11)->{din[9:8]==2'b10};
}

constraint mix_read_write{
    (prev_din==2'b00) -> {din[9:8] dist{2'b00:/50, 2'b01:/50}};
    //After a Write Data 60% → Read Address & 40% → Write Address
    (prev_din==2'b01) -> {din[9:8] dist{2'b10:/60, 2'b00:/40}};
    //Read address
    (prev_din==2'b10) -> {din[9:8] == 2'b11};
    //After a Read Data 60% → Write Address & 40% → Read
Address
    (prev_din==2'b11) -> {din[9:8] dist{2'b00:/60, 2'b10:/40}};
}

function void post_randomize();
    prev_din = din[9:8]; //To check the prev_din with the din now
in constraints
endfunction

endclass
endpackage

```

12. Sequencer of RAM:

```
package sequencer_pkg;
`include "uvm_macros.svh"
import seq_item_pkg::*;
import uvm_pkg::*;
class ram_sequencer extends uvm_sequencer #(ram_seq_item);
`uvm_component_utils(ram_sequencer);

function new(string name="ram_sequencer",uvm_component
parent=null);
super.new(name,parent);
endfunction
endclass
endpackage
```

13. Coverage collector of RAM:

```
package cov_pkg;
import uvm_pkg::*;
import seq_item_pkg::*;
import pkg_config::*;
`include "uvm_macros.svh"
class ram_coverage extends uvm_component;
`uvm_component_utils(ram_coverage)

virtual ram_interface ram_vif;
uvm_analysis_export #(ram_seq_item) cov_export;
uvm_tlm_analysis_fifo #(ram_seq_item) cov_fifo;
ram_seq_item seq_item_cov;
covergroup cvr_gp;

Transaction_ordering_cp: coverpoint seq_item_cov.din[9:8]
```

```

{
    bins possible_values={2'b00,2'b01,2'b10,2'b11};
    bins write_address_then_write_data=(2'b00 => 2'b01);
    bins read_address_then_read_data=(2'b10 => 2'b11);
    bins wr_address_then_wr_data_rd_address_then_rd_data=
(2'b00=>2'b01=>2'b10=>2'b11);
}
rx_valid_high_cp: coverpoint seq_item_cov.rx_valid
{bins rx_valid_high={1}};
tx_valid_high_cp: coverpoint seq_item_cov.tx_valid
iff(seq_item_cov.din[9:8]==2'b11)
{bins tx_valid_high={1}};

rx_valid_with_all_bins_din: cross
Transaction_ordering_cp,rx_valid_high_cp;
rd_data_with_tx_valid_high: cross
Transaction_ordering_cp,tx_valid_high_cp
{ignore_bins write_address_then_write_data_with_tx_valid_high=
binsof(Transaction_ordering_cp.write_address_then_write_data);
}

endgroup

function new(string name="alsu_scoreboard", uvm_component
parent =null) ;
super.new(name,parent);
cvr_gp=new();//create covergroup
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);
cov_export=new("cov_export",this);

```

```
cov_fifo=new("cov_fifo",this);
endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
cov_export.connect(cov_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);
forever begin
cov_fifo.get(seq_item_cov);
cvt_gp.sample();
end
endtask
```

14. Driver of RAM:

```
package pkg_driver;
import uvm_pkg::*;
import pkg_config::*;
import sequencer_pkg::*;
import seq_item_pkg::*;
import seq_pkg::*;
`include "uvm_macros.svh"
class ram_driver extends uvm_driver #(ram_seq_item);
`uvm_component_utils(ram_driver)

virtual ram_interface ram_vif;
ram_seq_item stim_seq_item;

function new(string name="ram_driver",uvm_component parent
=null) ;
super.new(name,parent);
endfunction

task run_phase(uvm_phase phase);
forever begin
    stim_seq_item=ram_seq_item::type_id::create("stim_seq_item");
    seq_item_port.get_next_item(stim_seq_item);
    //start of pulling data from sequencer

    ram_vif.rst_n=stim_seq_item.rst_n;
    ram_vif.din=stim_seq_item.din;
    ram_vif.rx_valid=stim_seq_item.rx_valid;
    @(negedge ram_vif.clk);

    seq_item_port.item_done(); //End of pilling
end
endtask
endclass
```

```

        `uvm_info("run_phase",stim_seq_item.convert2string_stimulus(),UVM_HIGH);
    end
endtask
endclass
endpackage

```

15. Monitor of RAM:

```

package mon_pkg;
`include "uvm_macros.svh"
import uvm_pkg::*;
import pkg_config::*;
import sequencer_pkg::*;
import seq_item_pkg::*;
class ram_monitor extends uvm_monitor;
`uvm_component_utils(ram_monitor)

virtual ram_interface ram_vif ;
ram_seq_item rsp_seq_item;
uvm_analysis_port #(ram_seq_item) mon_ap;

function new(string name="ram_monitor",uvm_component parent
=null) ;
super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
endfunction

```

```

endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);
forever begin
    rsp_seq_item=ram_seq_item::type_id::
create("rsp_seq_item",this);
    @(negedge ram_vif.clk)

    rsp_seq_item.rst_n=ram_vif.rst_n;
    rsp_seq_item.din=ram_vif.din;
    rsp_seq_item.rx_valid=ram_vif.rx_valid;
    rsp_seq_item.tx_valid=ram_vif.tx_valid;
    rsp_seq_item.dout=ram_vif.dout;

    mon_ap.write(rsp_seq_item);
    `uvm_info("run_phase",rsp_seq_item.convert2string(),
UVM_HIGH);
end

endtask
endclass
endpackage

```

16. Scoreboard of RAM:

```

package scoreboard_pkg;
`include "uvm_macros.svh"
import uvm_pkg::*;
import seq_item_pkg::*;
import seq_pkg::*;
class ram_scoreboard extends uvm_scoreboard;

```



```

`uvm_component_utils(ram_scoreboard)

uvm_analysis_export #(ram_seq_item) sb_export;
uvm_tlm_analysis_fifo #(ram_seq_item) sb_fifo;
ram_seq_item seq_item_sb;

int error_count=0;
int correct_count=0;
logic tx_valid_ref;
logic [7:0] dout_ref;

reg [7:0] MEM [255:0];
reg [7:0] Rd_Addr, Wr_Addr;

function new(string name="ram_scoreboard",uvm_component
parent=null);
super.new(name,parent);
endfunction

//BUILD ANALYSIS EXPORT AND FIFO
function void build_phase(uvm_phase phase);
super.build_phase(phase);
sb_export=new("sb_export",this);
sb_fifo=new("sb_fifo",this);
endfunction

//CONNECT ANALYSIS EXPORT
function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
sb_export.connect(sb_fifo.analysis_export);//Connecting fifo
with scoreboard
endfunction

```

```

task run_phase(uvm_phase phase);
super.run_phase(phase);
forever begin
sb_fifo.get(seq_item_sb);
ref_model(seq_item_sb);
if((seq_item_sb.dout!=dout_ref)&&(seq_item_sb.tx_valid!=tx_val
id_ref))begin
    `uvm_error("run_phase",$sformatf("comparison
failed,transaction recieved by the
DUT:%s",seq_item_sb.convert2string()));
        error_count++;
    end
    else begin
        `uvm_info("run_phase",$sformatf("correct out_refput:%s
",seq_item_sb.convert2string()),UVM_LOW);
            correct_count++;
        end
    end
end

endtask

task ref_model(ram_seq_item seq_item_chk);
if (!seq_item_chk.rst_n) begin
    dout_ref = 0;
    tx_valid_ref = 0;
    Rd_Addr = 0;
    Wr_Addr = 0;
end
else begin
    if (seq_item_chk.rx_valid) begin
        if (seq_item_chk.din[9:8]== 2'b00)
            Wr_Addr = seq_item_chk.din[7:0];
    end
end
end

```

```

        else if (seq_item_chk.din[9:8]== 2'b01)
            MEM[Wr_Addr] = seq_item_chk.din[7:0];
        else if (seq_item_chk.din[9:8]== 2'b10)
            Rd_Addr = seq_item_chk.din[7:0];
        else if (seq_item_chk.din[9:8]== 2'b11)
            dout_ref = MEM[Wr_Addr];
        else dout_ref = 0;
    end
    tx_valid_ref = (seq_item_chk.din[9] &&
seq_item_chk.din[8])? 1'b1 : 1'b0;
    end
endtask

function void report_phase (uvm_phase phase);
    super.report_phase(phase);
    `uvm_info("report_phase",$sformatf("Total successful
counts:%0d",correct_count),UVM_MEDIUM);
    `uvm_info("report_phase",$sformatf("Total failed
counts:%0d",error_count),UVM_MEDIUM);
endfunction
endclass
endpackage

```

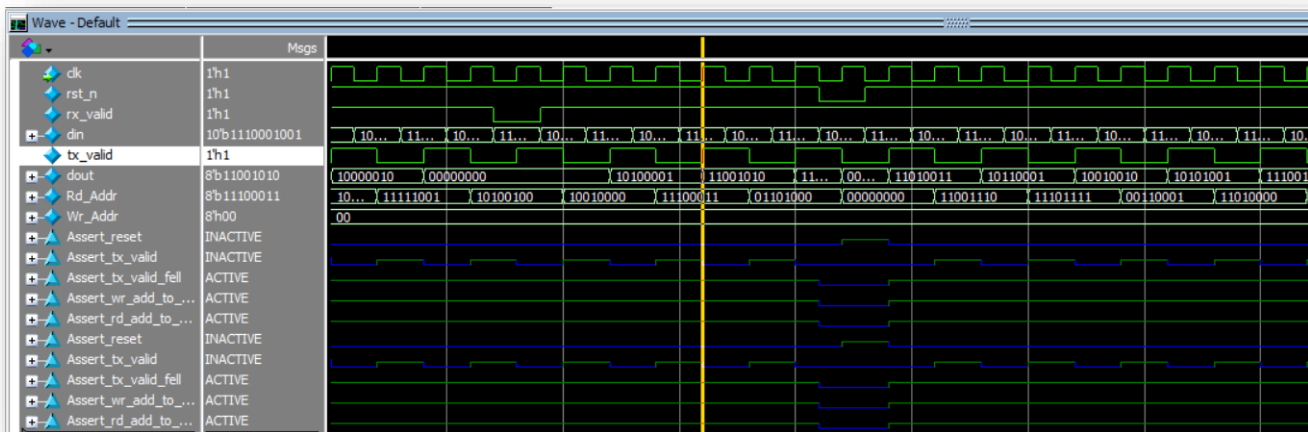
17. Do file of RAM:

```
vlib work
vlog -f ram/src_files.list\
+incdir+ram/uvvm\
+incdir+ram/rtl +cover -covercells

vsim -voptargs=+acc work.top -cover -classdebug -uvmmcontrol=all
coverage exclude -src ram/rtl/RAM.sv -line 20 -code b
coverage exclude -src ram/rtl/RAM.sv -line 20 -code s
coverage save ram.ucdb -onexit -du work.RAM
add wave -position insertpoint sim:/top/intf/*
add wave -position insertpoint sim:/top/DUT/*
add wave /top/DUT/binding_all/Assert_reset /top/DUT/binding_all/Assert_tx_valid /top/DUT/binding_all/Assert_tx_valid_fell /top/DUT/binding_all/Assert_wr_add_to_wr_data
/top/DUT/binding_all/Assert_rd_add_to_rd_data /top/assertion/Assert_reset /top/assertion/Assert_tx_valid /top/assertion/Assert_tx_valid_fell
/top/assertion/Assert_wr_add_to_wr_data /top/assertion/Assert_rd_add_to_rd_data
run -all
coverage report -detail -cvg -directive -comments -file F_cover_RAM.txt -noa -all
quit -sim
vcover report ram.ucdb -details -all -output coverage_rpt_RAM.txt
```

18. Simulation of RAM:

WAVE:



Assertions:

▲ /seq_pkg::write_only_sequence::body/#...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓
▲ /seq_pkg::read_only_sequence::body/#...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓
▲ /seq_pkg::write_read_sequence::body/#...	Immediate	SVA	on	0	1	-	-	-	-	off	assert (randomize(...))	✓
▲ /top/DUT/binding_all/Assert_reset	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) (~intf.rst...	✓
▲ /top/DUT/binding_all/Assert_tx_valid	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/DUT/binding_all/Assert_tx_valid_fell	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/DUT/binding_all/Assert_wr_add_to...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/DUT/binding_all/Assert_rd_add_to...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/assertion/Assert_reset	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) (~intf.rst...	✓
▲ /top/assertion/Assert_tx_valid	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/assertion/Assert_tx_valid_fell	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/assertion/Assert_wr_add_to_wr_data	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓
▲ /top/assertion/Assert_rd_add_to_rd_data	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0 off	assert(@(posedge intf.clk) disable if...	✓

Functional coverage:

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/cov_pkg/ram_coverage		100.0%							
TYPE cvr_gp		100.0%	100	100.0%					
CVP cvr_gp::Transaction_ordering_cp	ram_coverage	100.0%	100	100.0%					auto(0)
CVP cvr_gp::rx_valid_high_cp	ram_coverage	100.0%	100	100.0%					
CVP cvr_gp::tx_valid_high_cp	ram_coverage	100.0%	100	100.0%					
CROSS cvr_gp::rx_valid_with_all_bins_din	ram_coverage	100.0%	100	100.0%					
CROSS cvr_gp::rd_data_with_tx_valid_high	ram_coverage	100.0%	100	100.0%					
INST \cov_pkg::ram_coverage::cvr_gp		100.0%	100	100.0%					0
CVP Transaction_ordering_cp		100.0%	100	100.0%					
bin possible_values		30001	1	100.0%					
bin write_address_then_write_data		5735	1	100.0%					
bin read_address_then_read_data		5734	1	100.0%					
bin wr_address_then_wr_data_rd_address_then_rd_data		1047	1	100.0%					
CVP rx_valid_high_cp		100.0%	100	100.0%					
bin rx_valid_high		28471	1	100.0%					
CVP tx_valid_high_cp		100.0%	100	100.0%					
bin tx_valid_high		6807	1	100.0%					
CROSS rx_valid_with_all_bins_din		100.0%	100	100.0%					
bin <possible_values,rx_valid_high>		28471	1	100.0%					
bin <write_address_then_write_data,rx_valid_high>		5453	1	100.0%					
bin <read_address_then_read_data,rx_valid_high>		5448	1	100.0%					
bin <wr_address_then_wr_data_rd_address_then_rd_data,rx_valid_high>		994	1	100.0%					
CROSS rd_data_with_tx_valid_high		100.0%	100	100.0%					
bin <possible_values,tx_valid_high>		6807	1	100.0%					
bin <read_address_then_read_data,tx_valid_high>		5452	1	100.0%					
bin <wr_address_then_wr_data_rd_address_then_rd_data,tx_valid_high>		997	1	100.0%					
ignore_bin write_address_then_write_data_with_tx_valid_high		0	-	-					

19. Transcripts:

Reset:

Write only:

```
# START OF RESET
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 2: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0000000000, rx_valid=0b0, rst_n= 0b0, tx_valid= 0b0 ,dout= 0b00000000
# END OF RESET
```

```
# START OF WRITE ONLY
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 4: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0101001100, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 6: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0111010101, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 8: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0101100010, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 10: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0000111100, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 12: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0101000010, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 14: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0001110110, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 16: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0101011111, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 18: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0000011011, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 20: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0111010101, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 22: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0111001011, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 24: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0010011010, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 26: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0111110110, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 28: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0110100011, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 30: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0110111011, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM_INFO ram/uvm/scoreboard.sv(48) @ 32: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0100000110, rx_valid=0b0, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
# UVM INFO ram/uvm/scoreboard.sv(48) @ 34: uvm_test_top.env.sb [run_phase] correct out_refput: din=0b0101111001, rx_valid=0b1, rst_n= 0b1, tx_valid= 0b0 ,dout= 0b00000000
```


20. Report on RAM:

Statement coverage:

Branch coverage:

Coverage Report by file with details

```
====
File: ram/rtl/RAM.sv
=====
```

Statement Coverage:				
Enabled Coverage	Active	Hits	Misses	% Covered
Stmts	10	10	0	100.0

```
=====Statement Details=====
```

Statement Coverage for file ram/rtl/RAM.sv --

```
1      module RAM (ram_interface.DUT intf);
2
3      reg [7:0] MEM [255:0];
4      reg [7:0] Rd_Addr, Wr_Addr;
5
6          1      30001  always @(posedge intf.clk) begin
7              if (!intf.rst_n) begin
8                  1      1482      intf.dout <= 0;
9                  1      1482      intf.tx_valid <= 0;
10                 1      1482      Rd_Addr <= 0;
11                 1      1482      Wr_Addr <= 0;
12             end
13         else begin //put begin & end
14             if (intf.rx_valid) begin
15                 case (intf.din[9:8])
16                     1      6996      2'b00 : Wr_Addr <= intf.din[7:0];
17                     1      6553      2'b01 : MEM[Wr_Addr] <= intf.din[7:0];
18                     1      7033      2'b10 : Rd_Addr <= intf.din[7:0];
19                     1      6477      2'b11 : intf.dout <= MEM[Rd_Addr]; //bug MEM[Rd_Addr] instead of MEM[Wr_Addr]
20                     1      E      default : intf.dout <= 0;
21                 endcase
22             end
23             1      28519  intf.tx_valid <= (intf.din[9] && intf.din[8])? 1'b1 : 1'b0; //Deleting dependancy on rx_valid
24         end
25     end
26
27 endmodule
```

Branch Coverage:				
Enabled Coverage	Active	Hits	Misses	% Covered
Branches	8	8	0	100.0

```
=====Branch Details=====
```

Branch Coverage for file ram/rtl/RAM.sv --

```
-----IF Branch-----
7          30001  Count coming in to IF
7          1      1482      if (!intf.rst_n) begin
13         1      28519  else begin //put begin & end
Branch totals: 2 hits of 2 branches = 100.0%
```

```
-----IF Branch-----
14         28519  Count coming in to IF
14         1      27059  if (intf.rx_valid) begin
14         1460  All False Count
Branch totals: 2 hits of 2 branches = 100.0%
```

```
-----CASE Branch-----
15         27059  Count coming in to CASE
16         1      6996      2'b00 : Wr_Addr <= intf.din[7:0];
17         1      6553      2'b01 : MEM[Wr_Addr] <= intf.din[7:0];
18         1      7033      2'b10 : Rd_Addr <= intf.din[7:0];
19         1      6477      2'b11 : intf.dout <= MEM[Rd_Addr]; //bug MEM[Rd_Addr] instead of MEM[Wr_Addr]
20         1      E      default : intf.dout <= 0;
Branch totals: 4 hits of 4 branches = 100.0%
```

Condition coverage:

```

Condition Coverage:
  Enabled Coverage      Active   Covered   Misses % Covered
  -----
  FEC Condition Terms    0       0         0    100.0
Expression Coverage:
  Enabled Coverage      Active   Covered   Misses % Covered
  -----
  FEC Expression Terms    2       2         0    100.0

=====Expression Details=====

Expression Coverage for file ram/rtl/RAM.sv --

-----Focused Expression View-----
Line      23 Item    1  (intf.din[9] && intf.din[8])
Expression totals: 2 of 2 input terms covered = 100.0%

  Input Term   Covered   Reason for no coverage   Hint
  -----
  intf.din[9]      Y
  intf.din[8]      Y

  Rows:        Hits   FEC Target      Non-masking condition(s)
  -----
  Row  1:       1  intf.din[9]_0      -
  Row  2:       1  intf.din[9]_1      intf.din[8]
  Row  3:       1  intf.din[8]_0      intf.din[9]
  Row  4:       1  intf.din[8]_1      intf.din[9]

```

Toggle coverage:

```

FSM Coverage:
  Enabled Coverage      Active   Hits   Misses % Covered
  -----
  FSMs
    States              0       0         0    100.0
    Transitions         0       0         0    100.0
Toggle Coverage:
  Enabled Coverage      Active   Hits   Misses % Covered
  -----
  Toggle Bins          32      32         0    100.0

=====Toggle Details=====

Toggle Coverage for File ram/rtl/RAM.sv --

  Line      Node      1H->0L   0L->1H   "Coverage"
  -----
  4          Wr_Addr[7]    1         1    100.00
  4          Wr_Addr[6]    1         1    100.00
  4          Wr_Addr[5]    1         1    100.00
  4          Wr_Addr[4]    1         1    100.00
  4          Wr_Addr[3]    1         1    100.00
  4          Wr_Addr[2]    1         1    100.00
  4          Wr_Addr[1]    1         1    100.00
  4          Wr_Addr[0]    1         1    100.00
  4          Rd_Addr[7]    1         1    100.00
  4          Rd_Addr[6]    1         1    100.00
  4          Rd_Addr[5]    1         1    100.00
  4          Rd_Addr[4]    1         1    100.00
  4          Rd_Addr[3]    1         1    100.00
  4          Rd_Addr[2]    1         1    100.00
  4          Rd_Addr[1]    1         1    100.00
  4          Rd_Addr[0]    1         1    100.00

Total Node Count   =    16
Toggled Node Count =    16
Untoggled Node Count =    0
Toggle Coverage    = 100.0% (32 of 32 bins)

```


Assertion coverage:

```

DIRECTIVE COVERAGE:
-----
Name                               Design Design Lang File(Line)      Count Status
Unit   UnitType
-----
/\top#DUT /binding_all/cover_reset    SVA   Verilog  SVA  ram/rtl/Assertions.sv(6)
                                           1482 Covered
/\top#DUT /binding_all/cover_tx_valid  SVA   Verilog  SVA  ram/rtl/Assertions.sv(13)
                                           20627 Covered
/\top#DUT /binding_all/cover_tx_valid_fell
                                           SVA   Verilog  SVA  ram/rtl/Assertions.sv(19)
                                           2052 Covered
/\top#DUT /binding_all/cover_wr_add_to_wr_data
                                           SVA   Verilog  SVA  ram/rtl/Assertions.sv(25)
                                           16620 Covered
/\top#DUT /binding_all/cover_rd_add_to_rd_data
                                           SVA   Verilog  SVA  ram/rtl/Assertions.sv(31)
                                           16457 Covered

TOTAL DIRECTIVE COVERAGE: 100.0%  COVERS: 5

ASSERTION RESULTS:
-----
Name                               File(Line)          Failure Pass
                                Count   Count
-----
/\top#DUT /binding_all/Assert_reset    ram/rtl/Assertions.sv(5)      0    1
/\top#DUT /binding_all/Assert_tx_valid  ram/rtl/Assertions.sv(12)     0    1
/\top#DUT /binding_all/Assert_tx_valid_fell
                                ram/rtl/Assertions.sv(18)     0    1
/\top#DUT /binding_all/Assert_wr_add_to_wr_data
                                ram/rtl/Assertions.sv(24)     0    1
/\top#DUT /binding_all/Assert_rd_add_to_rd_data
                                ram/rtl/Assertions.sv(30)     0    1

Total Coverage By File (code coverage only, filtered view): 100.0%

```

```

DIRECTIVE COVERAGE:
-----
Name                               Design Design Lang File(Line)      Count Status
Unit   UnitType
-----
/top/DUT/binding_all/cover_reset    SVA   Verilog  SVA  ram/rtl/Assertions.sv(6)
                                           1482 Covered
/top/DUT/binding_all/cover_tx_valid  SVA   Verilog  SVA  ram/rtl/Assertions.sv(13)
                                           20627 Covered
/top/DUT/binding_all/cover_tx_valid_fell SVA   Verilog  SVA  ram/rtl/Assertions.sv(19)
                                           2052 Covered
/top/DUT/binding_all/cover_wr_add_to_wr_data
                                           SVA   Verilog  SVA  ram/rtl/Assertions.sv(25)
                                           16620 Covered
/top/DUT/binding_all/cover_rd_add_to_rd_data
                                           SVA   Verilog  SVA  ram/rtl/Assertions.sv(31)
                                           16457 Covered
/top/assertion/cover_reset          SVA   Verilog  SVA  ram/rtl/Assertions.sv(6)
                                           1482 Covered
/top/assertion/cover_tx_valid        SVA   Verilog  SVA  ram/rtl/Assertions.sv(13)
                                           20627 Covered
/top/assertion/cover_tx_valid_fell   SVA   Verilog  SVA  ram/rtl/Assertions.sv(19)
                                           2052 Covered
/top/assertion/cover_wr_add_to_wr_data
                                           SVA   Verilog  SVA  ram/rtl/Assertions.sv(25)
                                           16620 Covered
/top/assertion/cover_rd_add_to_rd_data
                                           SVA   Verilog  SVA  ram/rtl/Assertions.sv(31)
                                           16457 Covered

TOTAL DIRECTIVE COVERAGE: 100.0%  COVERS: 10

```

Cover groups:

COVERGROUP COVERAGE:			
Covergroup	Metric	Goal	Status
TYPE /cov_pkg/nam_coverage/cvr_gp	100.0%	100	Covered
covered/total bins:	13	13	
missing/total bins:	0	13	
% Hit:	100.0%	100	
Coverpoint cvr_gp::Transaction_ordering_cp	100.0%	100	Covered
covered/total bins:	4	4	
missing/total bins:	0	4	
% Hit:	100.0%	100	
Coverpoint cvr_gp::rx_valid_high_cp	100.0%	100	Covered
covered/total bins:	1	1	
missing/total bins:	0	1	
% Hit:	100.0%	100	
Coverpoint cvr_gp::tx_valid_high_cp	100.0%	100	Covered
covered/total bins:	1	1	
missing/total bins:	0	1	
% Hit:	100.0%	100	
Cross cvr_gp::rx_valid_with_all_bins_din	100.0%	100	Covered
covered/total bins:	4	4	
missing/total bins:	0	4	
% Hit:	100.0%	100	
Cross cvr_gp::rd_data_with_tx_valid_high	100.0%	100	Covered
covered/total bins:	3	3	
missing/total bins:	0	3	
% Hit:	100.0%	100	
CLASS nam_coverage			
Covergroup instance \cov_pkg::nam_coverage::cvr_gp	100.0%	100	Covered
covered/total bins:	13	13	
missing/total bins:	0	13	
% Hit:	100.0%	100	
Coverpoint Transaction_ordering_cp	100.0%	100	Covered
covered/total bins:	4	4	
missing/total bins:	0	4	
% Hit:	100.0%	100	
bin possible_values	30001	1	Covered
bin write_address_then_write_data	5735	1	Covered
bin read_address_then_read_data	5734	1	Covered
bin wr_address_then_wr_data_rd_address_then_rd_data	1047	1	Covered
Coverpoint rx_valid_high_cp	100.0%	100	Covered
covered/total bins:	1	1	
missing/total bins:	0	1	
% Hit:	100.0%	100	
bin rx_valid_high	28471	1	Covered
Coverpoint tx_valid_high_cp	100.0%	100	Covered
covered/total bins:	1	1	
missing/total bins:	0	1	
% Hit:	100.0%	100	
bin tx_valid_high	6807	1	Covered
Cross rx_valid_with_all_bins_din	100.0%	100	Covered
covered/total bins:	4	4	
missing/total bins:	0	4	
% Hit:	100.0%	100	
bin <possible_values,rx_valid_high>	28471	1	Covered
bin <write_address_then_write_data,rx_valid_high>	5453	1	Covered
bin <read_address_then_read_data,rx_valid_high>	5448	1	Covered
bin <wr_address_then_wr_data_rd_address_then_rd_data,rx_valid_high>	994	1	Covered
Cross rd_data_with_tx_valid_high	100.0%	100	Covered
covered/total bins:	3	3	
missing/total bins:	0	3	
% Hit:	100.0%	100	
bin <possible_values,tx_valid_high>	6807	1	Covered
bin <read_address_then_read_data,tx_valid_high>	5452	1	Covered
bin <wr_address_then_wr_data_rd_address_then_rd_data,tx_valid_high>	997	1	Covered
ignore_bin write_address_then_write_data_with_tx_valid_high	0		ZERO
TOTAL COVERGROUP COVERAGE: 100.0% COVERGROUP TYPES: 1			

21. Assertions Table

Feature	Assertion
At positive edge clock when reset is asserted	@(posedge clk) (!rst_n) => (tx_valid==1'b0 && dout==8'b0);
At positive edge clock disable if reset asserted if (write address or write data or read address)	@(posedge clk) disable iff(!rst_n) (din[9:8]==2'b00 din[9:8]==2'b01 din[9:8]==2'b10) => (tx_valid==0);
At positive edge clock (disable if reset asserted) when read data operation	@(posedge clk) disable iff(!rst_n) (din[9:8]==2'b11) => (tx_valid==1) => (din[9:8]==2'b00 din[9:8]==2'b10) => (tx_valid==0) ;
At positive edge clock (disable if reset asserted) if write address operation	@(posedge clk) disable iff(!rst_n) (din[9:8]==2'b00) [=1:\$] ##1 (din[9:8]==2'b01);
At positive edge clock (disable if reset asserted) if read address operation	@(posedge clk) disable iff(!rst_n) (din[9:8]==2'b10) [=1:\$] ##1 (din[9:8]==2'b11);

22. Bug Report:

- Inside MEM was Wr_Addr so that's wrong because in read case we read the the read address that we saved in the previous operation

Solution: dout <= MEM[Rd_Addr]; //bug MEM[Rd_Addr] instead of MEM[Wr_Addr]

- this line should be dependent on rx_valid and updated after each case

Solution: tx_valid <= (din[9] && din[8])? 1'b1 : 1'b0;

c. Part 3: UVM Environment for SPI Wrapper

1. Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functional Checking
Wrapper_1	When the reset is Deasserted, the output MISO should be held LOW	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.	NA	A golden Model reference Checker in the scoreboard to make sure the output is correct, p_reset_MISO_assertion
Wrapper_2	In case of rst_n is high, Write only sequence by always setting the MOSI first 3 bits to b000 or b001	Randomization under constraints "write_only_c" on MOSI where every Write Address operation shall always be followed by either Write Address or Write Data operation.	Covers: cp_MOSI	p_MISO_stable_assertion
Wrapper_3	In case of rst_n is high, Read only sequence by always setting the MOSI first 3 bits to b110 or b111	Randomization under constraints "read_only_c" on MOSI where every Read Address operation shall always be followed by Read Data. After a Read Data operation shall always be followed by Read Address..	Covers: cp_MOSI	A golden Model reference Checker in the scoreboard to make sure the output is correct, p_MISO_stable_assertion
Wrapper_4	In case of rst_n is high, Read and Write sequence by always setting the MOSI first 3 bits to b110 or b111 or b000 or b001	Randomization under constraints "read_write_c" on MOSI where every operation has a certain pool of next operation possible	Covers: cp_MOSI	A golden Model reference Checker in the scoreboard to make sure the output is correct, p_MISO_stable_assertion
Wrapper_5	In case of rst_n is high, SS_n should be held low in case of communication and High in case of no communication	Directed to be low for 23 clock cycles in case of Read operation and 13 in case of other valid operations	covers: cp_SS_n	NA

2. Top Module

```
1 import wrapper_test_pkg::*;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 module top();
5
6     bit clk;
7
8     initial begin
9         clk = 0;
10         forever
11             #1 clk = ~clk;
12     end
13     //wrapper interface Instance
14     wrapper_if wrapperif (clk);
15     //ram interface instance
16     ram_interface ramif(clk);
17     //slave interface instance
18     SPI_if slaveif(clk);
19
20     WRAPPER DUT (.MOSI(wrapperif.MOSI), .MISO(wrapperif.MISO), .SS_n(wrapperif.SS_n),
21                 .clk(clk), .rst_n(wrapperif.rst_n));
22
23     wrapper_golden golden_model(.MOSI(wrapperif.MOSI), .MISO(wrapperif.MISO_golden), .SS_n(wrapperif.SS_n),
24                                 .clk(clk), .rst_n(wrapperif.rst_n));
25
26     //bind the SVA module to the designs
27     bind WRAPPER wrapper_SVA wrapper_SVA_inst(.MOSI(wrapperif.MOSI), .MISO(wrapperif.MISO), .SS_n(wrapperif.SS_n),
28                                                .rx_valid(DUT.rx_valid), .rx_data(DUT.rx_data_din) , .tx_valid(DUT.tx_valid),
29                                                .clk(clk), .rst_n(wrapperif.rst_n));
30
31     //Connect interface of the RAM and Salve to DUT internal signals
32     //-----
33     //Slave Interface connections
34     //-----
35     assign slaveif.MOSI = DUT.MOSI;
36     assign slaveif.rst_n = DUT.rst_n;
37     assign slaveif.SS_n = DUT.SS_n;
38     assign slaveif.MISO = DUT.MISO;
39     assign slaveif.MISO_golden = golden_model.MISO;
40     assign slaveif.tx_data = DUT.tx_data_dout;
41     assign slaveif.tx_valid = DUT.tx_valid;
42     assign slaveif.rx_data = DUT.rx_data_din;
43     assign slaveif.rx_valid = DUT.rx_valid;
44     assign slaveif.rx_data_golden = golden_model.rx_data;
45     assign slaveif.rx_valid_golden = golden_model.rx_valid;
46
47     //-----
48     // Ram Interface connections
49     //-----
50     assign ramif.rst_n = DUT.rst_n;
51     assign ramif.dout = DUT.tx_data_dout;
52     assign ramif.tx_valid = DUT.tx_valid;
53     assign ramif.din = DUT.rx_data_din;
54     assign ramif.rx_valid = DUT.rx_valid;
55
56     initial begin
57         uvm_config_db#(virtual wrapper_if)::set(null, "uvm_test_top", "wrapper_vif", wrapperif);
58         //Set the interface in configuration data base for RAM and Slave
59         uvm_config_db#(virtual ram_interface)::set(null, "uvm_test_top", "ram_vif", ramif);
60         uvm_config_db#(virtual SPI_if)::set(null, "uvm_test_top", "slave_vif", slaveif);
61         $readmemb("mem.dat", DUT.RAM_instance.MEM );//initializing memory
62         run_test("wrapper_test");
63     end
64
65 endmodule
```

3. Wrapper Interface

```
1 interface wrapper_if(clk);
2     input clk;
3     //Inputs
4     logic MOSI, SS_n, clk, rst_n;
5     //outputs
6     logic MISO;
7     //Golden Outputs
8     logic MISO_golden;
9 endinterface : wrapper_if
```

4. Wrapper Assertions

```
1 module wrapper_SVA(MOSI, MISO, SS_n, rx_valid, rx_data, tx_valid , clk, rst_n);
2
3     parameter INACTIVE = 0;
4
5     input logic MOSI, SS_n, rx_valid, tx_valid, clk, rst_n;
6     input logic [9:0] rx_data;
7     input logic MISO;
8
9
10    //1.Reset behavior
11    property p_reset_MISO;
12        @(posedge clk) (~rst_n) | => (MISO == INACTIVE);
13    endproperty
14
15    property p_reset_rx_valid;
16        @(posedge clk) (~rst_n) | => (rx_valid == INACTIVE);
17    endproperty
18
19    property p_reset_rx_data;
20        @(posedge clk) (~rst_n) | => (rx_data == INACTIVE);
21    endproperty
22
23    //2.Stable MISO in case of non-read operation
24    property p_MISO_stable;
25        @(posedge clk) disable iff (~rst_n) !tx_valid | => (MISO == $stable(MISO) [->1]);
26    endproperty
27
28    //-----
29    // Assertions
30    //-----
31    p_reset_MISO_assertion          : assert property (p_reset_MISO);
32    p_reset_rx_valid_assertion      : assert property (p_reset_rx_valid);
33    p_reset_rx_data_assertion       : assert property (p_reset_rx_data);
34    p_MISO_stable_assertion         : assert property (p_MISO_stable);
35
36
37
38    //-----
39    // Coverage
40    //-----
41    p_reset_MISO_coverage          : cover property (p_reset_MISO);
42    p_reset_rx_valid_coverage      : cover property (p_reset_rx_valid);
43    p_reset_rx_data_coverage       : cover property (p_reset_rx_data);
44    p_MISO_stable_coverage         : cover property (p_MISO_stable);
45
46
47 endmodule
```

5. Test Code

```
1 package wrapper_test_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5
6 import wrapper_env_pkg::*;
7 import wrapper_config_pkg::*;
8 import wrapper_seq_pkg::*;
9
10 //Import Ram Env & Config pkg
11 import pkg_env::*;
12 import pkg_config::*;
13
14 //Import Slave Env & Config pkg
15 import SPI_env_pkg::*;
16 import SPI_config_pkg::*;
17
18 class wrapper_test extends uvm_test;
19     `uvm_component_utils(wrapper_test)
20
21     //Wrapper
22     wrapper_env my_wrapper_env;
23     wrapper_config wrapper_config_obj_test;
24
25     //Slave
26     SPI_env my_slave_env;
27     SPI_config SPI_config_obj_test;
28
29     //RAM
30     ram_env my_ram_env;
31     ram_config_obj ram_config_obj_test;
32
33     //Sequences
34     wrapper_reset_seq reset_seq;
35     wrapper_write_only_sequence write_only_seq;
36     wrapper_read_only_sequence read_only_seq;
37     wrapper_write_read_sequence write_read_seq;
38
39     function new(string name = "wrapper_test", uvm_component parent = null);
40         super.new(name, parent);
41     endfunction
42
43     function void build_phase(uvm_phase phase);
44         super.build_phase(phase);
45
46         my_wrapper_env = wrapper_env::type_id::create("my_wrapper_env", this);
47         my_ram_env = ram_env::type_id::create("my_ram_env", this);
48         my_slave_env = SPI_env::type_id::create("my_slave_env", this);
49
50         reset_seq = wrapper_reset_seq::type_id::create("reset_seq");
51         write_only_seq = wrapper_write_only_sequence::type_id::create("write_only_seq");
52         read_only_seq = wrapper_read_only_sequence::type_id::create("read_only_seq");
53         write_read_seq = wrapper_write_read_sequence::type_id::create("write_read_seq");
54
55         wrapper_config_obj_test = wrapper_config::type_id::create("wrapper_config_obj_test");
56         ram_config_obj_test = ram_config_obj::type_id::create("ram_config_obj_test");
57         SPI_config_obj_test = SPI_config::type_id::create("SPI_config_obj_test");
58         if(!uvm_config_db #(virtual wrapper_if)::get(this, "", "wrapper_vif", wrapper_config_obj_test.wrapper_config_vif))
59             `uvm_fatal("build_phase", "Test - Unable to get WRAPPER Virtual Interface");
60
61         //Get Config object for ram
62         if(!uvm_config_db #(virtual ram_interface)::get(this, "", "ram_vif", ram_config_obj_test.ram_vif))
63             `uvm_fatal("build_phase", "Test - Unable to get RAM Virtual Interface");
64
65         //Get Config object for slave
66         if(!uvm_config_db #(virtual SPI_if)::get(this, "", "slave_vif", SPI_config_obj_test.SPI_vif))
67             `uvm_fatal("build_phase", "Test - Unable to get SLAVE Virtual Interface");
68
69         wrapper_config_obj_test.is_active = UVM_ACTIVE;
70         ram_config_obj_test.is_active = UVM_PASSIVE;
71         SPI_config_obj_test.is_active = UVM_PASSIVE;
72
73         uvm_config_db #(wrapper_config)::set(null, "", "WRAPPER_CFG", wrapper_config_obj_test);
74         //set configuration object in config data base for ram
75         uvm_config_db #(ram_config_obj)::set(null, "", "RAM_CFG", ram_config_obj_test);
76         //set configuration object in config data base for slave
77         uvm_config_db #(SPI_config)::set(null, "", "SLAVE_CFG", SPI_config_obj_test);
78     endfunction
```



```

81
82     task run_phase(uvm_phase phase);
83         super.run_phase(phase);
84         phase.raise_objection(this);
85
86         `uvm_info("run_phase", "Reset Asserted", UVM_MEDIUM);
87         reset_seq.start(my_wrapper_env.agt.sqr);
88         `uvm_info("run_phase", "Reset Deasserted", UVM_MEDIUM);
89
90         `uvm_info("run_phase", "Write only Sequencer Begin", UVM_MEDIUM);
91         write_only_seq.start(my_wrapper_env.agt.sqr);
92         `uvm_info("run_phase", "Write only Sequencer end", UVM_MEDIUM);
93
94         `uvm_info("run_phase", "Read only Sequencer Begin", UVM_MEDIUM);
95         read_only_seq.start(my_wrapper_env.agt.sqr);
96         `uvm_info("run_phase", "Read only Sequencer end", UVM_MEDIUM);
97
98         `uvm_info("run_phase", "Write & Read Sequencer Begin", UVM_MEDIUM);
99         write_read_seq.start(my_wrapper_env.agt.sqr);
100         `uvm_info("run_phase", "Write & Read Sequencer end", UVM_MEDIUM);
101
102         phase.drop_objection(this);
103     endtask
104 endclass
105
106 endpackage

```

```

1 package wrapper_env_pkg;
2
3     import uvm_pkg::*;
4     `include "uvm_macros.svh"
5
6     import wrapper_agent_pkg::*;
7     import wrapper_scoreboard_pkg::*;
8     import wrapper_covcollector_pkg::*;
9
10    class wrapper_env extends uvm_env;
11        `uvm_component_utils(wrapper_env);
12
13        wrapper_agent agt;
14        wrapper_scoreboard sb;
15        wrapper_covcollector cov;
16
17        function new(string name = "wrapper_env", uvm_component parent = null);
18            super.new(name, parent);
19        endfunction
20
21        function void build_phase(uvm_phase phase);
22            super.build_phase(phase);
23            agt = wrapper_agent::type_id::create("agt", this);
24            sb = wrapper_scoreboard::type_id::create("sb", this);
25            cov = wrapper_covcollector::type_id::create("cov", this);
26        endfunction
27
28        function void connect_phase(uvm_phase phase);
29            super.connect_phase(phase);
30            agt.agt_ap.connect(sb.sb_export);
31            agt.agt_ap.connect(cov.cov_export);
32        endfunction
33
34    endclass
35
36 endpackage

```


7. Wrapper Configuration object

```
1 package wrapper_config_pkg;
2     import uvm_pkg::*;
3     `include "uvm_macros.svh"
4
5     class wrapper_config extends uvm_object;
6         `uvm_object_utils(wrapper_config)
7         // Configuration parameters
8         virtual wrapper_if wrapper_config_vif;
9         uvm_active_passive_enum is_active;
10
11         function new(string name = "wrapper_config");
12             super.new(name);
13         endfunction
14     endclass : wrapper_config
15 endpackage
```

8. Wrapper

```
1 package wrapper_seq_item_pkg;
2
3     import uvm_pkg::*;
4     `include "uvm_macros.svh"
5     class wrapper_seq_item extends uvm_sequence_item;
6         `uvm_object_utils(wrapper_seq_item);
7
8         rand bit rst_n;
9         rand bit SS_n;
10
11         rand bit MOSI;
12
13         bit MISO_golden;
14         bit MISO;
15
16         typedef enum bit [2:0] {WRITE_ADDR = 3'b000, WRITE_DATA = 3'b001, READ_ADDR = 3'b110, READ_DATA = 3'b111} e_state;
17
18         rand e_state ns;
19         e_state cs;
20
21         int counter; // Counter for Serialization
22         int SS_n_cnt; // Counter for Communication
23         int WIDTH; // WIDTH = 13 (cs != READ_DATA) / 23 (cs = READ_DATA)
24
25         rand bit MOSI_BUS_RAND[];
26
27         bit MOSI_BUS[]; // [0:1:2]
28
29
30
31         constraint rst_n_c { rst_n dist { 0:/2 , 1:/98 }; }
32
33         constraint read_write_c {
34             if (!rst_n) {
35                 ns == e_state'(0);
36             }
37             else {
38                 ns inside {WRITE_ADDR,WRITE_DATA,READ_ADDR,READ_DATA};
39                 if (cs == WRITE_ADDR)
40                     ns inside {WRITE_ADDR, WRITE_DATA};
41                 else if (cs == WRITE_DATA)
42                     ns dist {WRITE_ADDR :/40, READ_ADDR :/60};
43                 else if (cs == READ_ADDR)
44                     ns == READ_DATA;
45                 else if (cs == READ_DATA)
46                     ns dist {WRITE_ADDR :/60, READ_ADDR :/40};
47             }
48         }
```

```

1
2  constraint read_only_c{
3      if (!rst_n){
4          ns == e_state'(0);
5      }
6      else{
7          ns inside {READ_ADDR, READ_DATA};
8          if(cs == READ_ADDR)
9              ns == READ_DATA;
10         else{
11             ns == READ_ADDR;
12         }
13     }
14 }
15
16 constraint write_only_c{
17     if(!rst_n){
18         ns == e_state'(0);
19     }
20     else{
21         ns inside {WRITE_ADDR, WRITE_DATA};
22         if(cs == WRITE_ADDR)
23             ns == WRITE_DATA;
24         else{
25             ns inside {WRITE_ADDR, WRITE_DATA};
26         }
27     }
28 }
29
30
31 constraint mosi_c {
32     if (!rst_n) {
33         MOSI == 0;
34     }
35     else {
36         if (SS_n_cnt != WIDTH)           // MSB      LSB
37             MOSI == MOSI_BUS[counter]; // [0:1:2...WIDTH]
38     }
39 }
40
41 constraint ss_n_c {
42     if (!rst_n) {
43         SS_n == 1;
44     }
45     else {
46         if (cs != READ_DATA) {
47             if (SS_n_cnt < 13)
48                 SS_n == 0;
49             else
50                 SS_n == 1;
51         }
52         else {
53             if (SS_n_cnt < 23)
54                 SS_n == 0;
55             else
56                 SS_n == 1;
57         }
58     }
59 }
60

```

```

1  // Responsible for Serialization Counter (counter) & cs reloading
2  function void pre_randomize ();
3      if (!rst_n) begin
4          counter = 0;
5      end
6      else begin
7          if (SS_n_cnt == 0) begin // OFF COMM
8              counter = 0;
9              cs = ns;
10
11              if (cs != READ_DATA) begin
12                  WIDTH = 13;
13                  MOSI_BUS = new[WIDTH];
14                  MOSI_BUS_RAND = new[WIDTH];
15              end
16              else begin
17                  WIDTH = 23;
18                  MOSI_BUS = new[WIDTH];
19                  MOSI_BUS_RAND = new[WIDTH];
20              end
21          end
22          else begin
23              // cs constant
24              counter = counter + 1;
25          end
26      end
27  endfunction
28
29  // Responsible for Communication Counter (ss_n_cnt)
30  function void post_randomize ();
31      if (!rst_n) begin
32          SS_n_cnt = 0;
33      end
34      else begin
35          MOSI_BUS = MOSI_BUS_RAND;
36          for (int i = 0 ; i < 3 ; i++) begin
37              MOSI_BUS [i] = cs[2-i]; // MOSI_BUS[0:2] = cs;
38          end
39
40          if ((cs != READ_DATA) && (SS_n_cnt != 13))
41              SS_n_cnt += 1;
42          else if ((cs == READ_DATA) && (SS_n_cnt != 23))
43              SS_n_cnt += 1;
44          else
45              SS_n_cnt = 0;
46      end
47  endfunction
48
49
50  function new (string name="wrapper_seq_item") ;
51      super.new(name);
52  endfunction
53
54
55  endclass
56  endpackage

```

9. Wrapper Sequencer

10.

```
1 package wrapper_sequencer_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5 import wrapper_seq_item_pkg::*;
6
7 class wrapper_sequencer extends uvm_sequencer #(wrapper_seq_item);
8     `uvm_component_utils(wrapper_sequencer)
9     function new(string name = "alsu_sequencer", uvm_component parent = null);
10         super.new(name, parent);
11     endfunction //new()
12 endclass //wrapper_sequencer extends uvm_sequencer
13 endpackage
```

```
1 package wrapper_agent_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5 import wrapper_driver_pkg::*;
6 import wrapper_sequencer_pkg::*;
7 import wrapper_monitor_pkg::*;
8 import wrapper_config_pkg::*;
9 import wrapper_seq_item_pkg::*;
10
11
12 class wrapper_agent extends uvm_agent;
13     `uvm_component_utils(wrapper_agent)
14
15     wrapper_config wrapper_cfg;
16     wrapper_driver drv;
17     wrapper_monitor mon;
18     wrapper_sequencer sqr;
19     uvm_analysis_port #(wrapper_seq_item) agt_ap;
20
21
22     function new(string name = "wrapper_agent", uvm_component parent = null);
23         super.new(name, parent);
24     endfunction //new()
25
26     function void build_phase(uvm_phase phase);
27         super.build_phase(phase);
28         if(!uvm_config_db #(wrapper_config)::get(this, "", "WRAPPER_CFG", wrapper_cfg))begin
29             `uvm_fatal("build_phase", "Error, Unable to get configuration object")
30         end
31         if(wrapper_cfg.is_active == UVM_ACTIVE)begin
32             drv = wrapper_driver::type_id::create("drv", this);
33             sqr = wrapper_sequencer::type_id::create("sqr", this);
34         end
35         mon = wrapper_monitor::type_id::create("mon", this);
36         agt_ap = new("agt_ap", this);
37     endfunction
38
39     function void connect_phase(uvm_phase phase);
40         super.connect_phase(phase);
41         //driver connections
42         if(wrapper_cfg.is_active == UVM_ACTIVE)begin
43             drv.wrapper_vif = wrapper_cfg.wrapper_config_vif;
44             drv.seq_item_port.connect(sqr.seq_item_export);
45         end
46         //monitor connections
47         mon.wrapper_vif = wrapper_cfg.wrapper_config_vif;
48         mon.mon_ap.connect(agt_ap);
49     endfunction
50 endclass //wrapper_agent extends uvm_agent
51
52 endpackage
```

11. Wrapper Driver

```
1 package wrapper_driver_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5 import wrapper_seq_item_pkg::*;
6
7
8 class wrapper_driver extends uvm_driver #(wrapper_seq_item);
9     `uvm_component_utils(wrapper_driver);
10
11     virtual wrapper_vif wrapper_vif;
12     wrapper_seq_item stim_seq_item;
13
14
15     function new(string name = "wrapper_driver", uvm_component parent);
16         super.new(name, parent);
17     endfunction
18
19
20     task run_phase(uvm_phase phase);
21         super.run_phase(phase);
22         forever begin
23             stim_seq_item = wrapper_seq_item::type_id::create("stim_seq_item");
24             seq_item_port.get_next_item(stim_seq_item);
25             wrapper_vif.rst_n = stim_seq_item.rst_n;
26             wrapper_vif.SS_n = stim_seq_item.SS_n;
27             wrapper_vif.MOSI = stim_seq_item.MOSI;
28             @(negedge wrapper_vif.clk);
29             seq_item_port.item_done();
30         end
31     endtask
32
33
34 endclass
35
36 endpackage
```

12. Wrapper Monitor

```
1 package wrapper_monitor_pkg;
2
3     import uvm_pkg::*;
4     `include "uvm_macros.svh"
5     import wrapper_seq_item_pkg::*;
6
7
8     class wrapper_monitor extends uvm_monitor;
9         `uvm_component_utils(wrapper_monitor);
10
11         virtual wrapper_if wrapper_vif;
12         wrapper_seq_item rsp_seq_item;
13         uvm_analysis_port #(wrapper_seq_item) mon_ap;
14
15
16         function new(string name = "wrapper_monitor", uvm_component parent);
17             super.new(name, parent);
18         endfunction
19
20         function void build_phase(uvm_phase phase);
21             super.build_phase(phase);
22             mon_ap = new("mon_ap", this);
23         endfunction
24
25
26         task run_phase(uvm_phase phase);
27             super.run_phase(phase);
28             forever begin
29                 rsp_seq_item = wrapper_seq_item::type_id::create("rsp_seq_item");
30                 @(negedge wrapper_vif.clk);
31                 rsp_seq_item.rst_n = wrapper_vif.rst_n;
32                 rsp_seq_item.ss_n = wrapper_vif.ss_n;
33                 rsp_seq_item.MOSI = wrapper_vif.MOSI;
34                 rsp_seq_item.MISO = wrapper_vif.MISO;
35                 rsp_seq_item.MISO_golden = wrapper_vif.MISO_golden;
36                 mon_ap.write(rsp_seq_item);
37             end
38         endtask
39     endclass
40
41 endpackage
```

13. Wrapper Scoreboard

```
1 package wrapper_scoreboard_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5
6 import wrapper_seq_item_pkg::*;
7
8
9 class wrapper_scoreboard extends uvm_scoreboard;
10     `uvm_component_utils(wrapper_scoreboard)
11
12     uvm_analysis_export #(wrapper_seq_item) sb_export;
13     uvm_tlm_analysis_fifo #(wrapper_seq_item) sb_fifo;
14     wrapper_seq_item seq_item;
15
16     int error_count = 0;
17     int correct_count = 0;
18
19     function new(string name = "wrapper_scoreboard", uvm_component parent = null);
20         super.new(name, parent);
21     endfunction //new()
22
23     function void build_phase(uvm_phase phase);
24         super.build_phase(phase);
25         sb_export = new("sb_export", this);
26         sb_fifo = new("sb_fifo", this);
27     endfunction
28
29     function void connect_phase(uvm_phase phase);
30         super.connect_phase(phase);
31         sb_export.connect(sb_fifo.analysis_export);
32     endfunction
33
34     task run_phase(uvm_phase phase);
35         super.run_phase(phase);
36         forever begin
37             sb_fifo.get(seq_item);
38             if(seq_item.MISO != seq_item.MISO_golden)begin
39                 `uvm_error("run_phase", $sformatf("Mismatch in Output | Expected: %b | DUT: %b", seq_item.MISO_golden, seq_item.MISO));
40                 error_count++;
41             end
42             else begin
43                 correct_count++;
44             end
45         end
46     endtask
47
48     function void report_phase(uvm_phase phase);
49         super.report_phase(phase);
50         `uvm_info("report_phase", $sformatf("Wrapper: total successful transactions: %d", correct_count),UVM_MEDIUM);
51         `uvm_info("report_phase", $sformatf("Wrapper: total Failed transactions: %d", error_count),UVM_MEDIUM);
52     endfunction
53
54     endclass //wrapper_scoreboard extends uvm_scoreboard
55
56 endpackage
```

14. Wrapper Coverage Collector

```
1  package wrapper_CovCollector_pkg;
2
3      import uvm_pkg::*;
4      `include "uvm_macros.svh"
5
6      import wrapper_seq_item_pkg::*;
7
8      class wrapper_covcollector extends uvm_component;
9          `uvm_component_utils(wrapper_covcollector)
10
11          uvm_analysis_export #(wrapper_seq_item) cov_export;
12          uvm_tlm_analysis_fifo #(wrapper_seq_item) cov_fifo;
13          wrapper_seq_item seq_item;
14
15
16          covergroup CovCode;
17
18              cp_SS_n: coverpoint seq_item.SS_n{
19                  bins normal_txn = (1=> 0[*13] => 1);
20                  bins extended_txn = (1=> 0[*23] => 1);
21              }
22
23              cp_MOSI: coverpoint seq_item.MOSI iff (seq_item.rst_n && !seq_item.SS_n){
24                  bins write_addr = (0=>0=>0);
25                  bins write_data = (0=>0=>1);
26                  bins read_addr  = (1=>1=>0);
27                  bins read_data  = (1=>1=>1);
28              }
29
30          endgroup
31
32          function new(string name = "wrapper_covcollector", uvm_component parent = null);
33              super.new(name, parent);
34              CovCode = new;
35          endfunction //new()
36
37          function void build_phase(uvm_phase phase);
38              super.build_phase(phase);
39              cov_export = new("cov_export", this);
40              cov_fifo = new("cov_fifo", this);
41          endfunction
42
43          function void connect_phase(uvm_phase phase);
44              super.connect_phase(phase);
45              cov_export.connect(cov_fifo.analysis_export);
46          endfunction
47
48          task run_phase(uvm_phase phase);
49              super.run_phase(phase);
50              forever begin
51                  cov_fifo.get(seq_item);
52                  CovCode.sample();
53              end
54          endtask
55
56          endclass //wrapper_CovCollector extends uvm_component
57
58      endpackage
```


15. Coverage

Functional Coverage:

/wrapper_CovCollector_pkg/wrapper_covcollector	100.00%								
TYPE CovCode	100.00%	100	100.00...						✓
CVP CovCode::cp_SS_n	100.00%	100	100.00...						✓
CVP CovCode::cp_MOSI	100.00%	100	100.00...						✓
INST \wrapper_CovCollector_pkg::wrapp...	100.00%	100	100.00...						✓
CVP cp_SS_n	100.00%	100	100.00...						✓
bin normal_txn		131	1	100.00...					✓
bin extended_txn		27	1	100.00...					✓
CVP cp_MOSI	100.00%	100	100.00...						✓
bin write_addr		475	1	100.00...					✓
bin write_data		365	1	100.00...					✓
bin read_addr		314	1	100.00...					✓
bin read_data		331	1	100.00...					✓

Assertion Coverage:

/top/DUT/wrapper_SVA_inst/p_reset_MISO_coverage	SVA	✓	Off	62	1 Unli...	1	100%	
/top/DUT/wrapper_SVA_inst/p_reset_rx_valid_coverage	SVA	✓	Off	62	1 Unli...	1	100%	
/top/DUT/wrapper_SVA_inst/p_reset_rx_data_coverage	SVA	✓	Off	62	1 Unli...	1	100%	
/top/DUT/wrapper_SVA_inst/p_MISO_stable_coverage	SVA	✓	Off	260	1 Unli...	1	100%	

Coverage Report:

Recursive Coverage Report Summary Data by DU

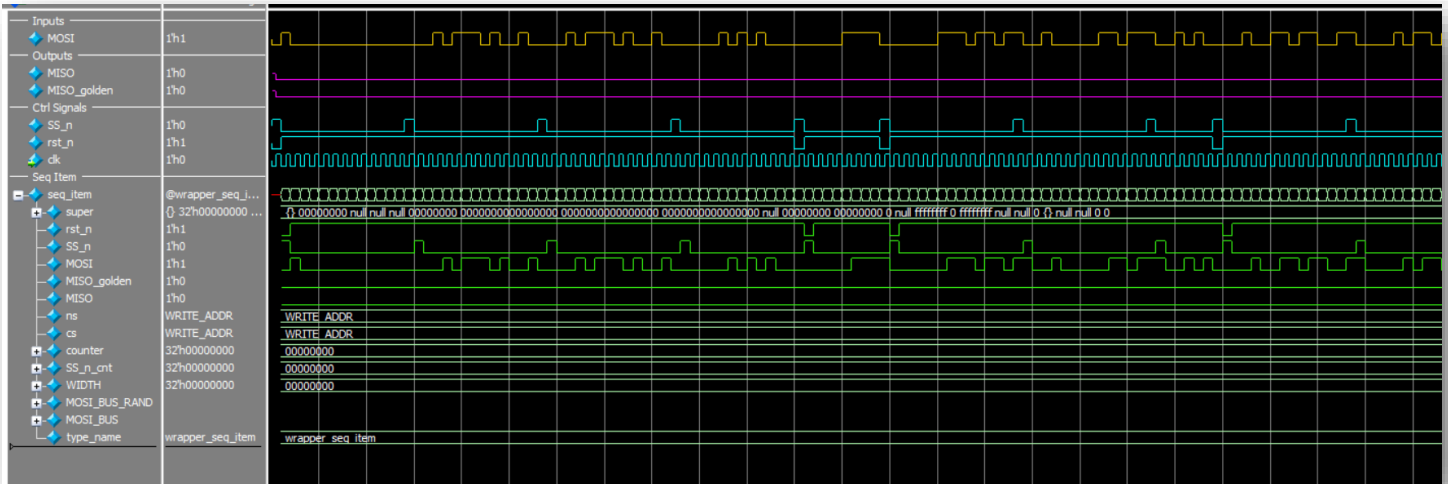
=====				
=== Design Unit: work.WRAPPER				
=====				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	----	-----
Assertions	4	4	0	100.00%
Branches	49	47	2	95.91%
Conditions	4	4	0	100.00%
Directives	4	4	0	100.00%
FSM States	5	5	0	100.00%
FSM Transitions	8	8	0	100.00%
Statements	52	52	0	100.00%
Toggles	232	232	0	100.00%

Total Coverage By Design Unit (filtered view): 99.48%

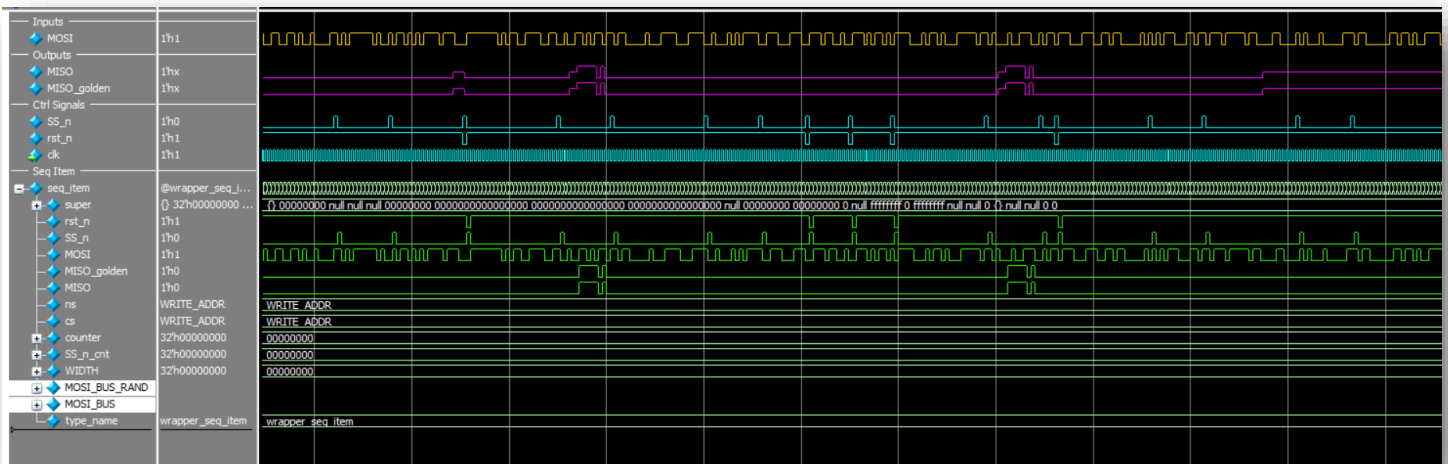
Note: Branch coverage not 100% due to default statement of case statement

16. QuestaSim Waveforms

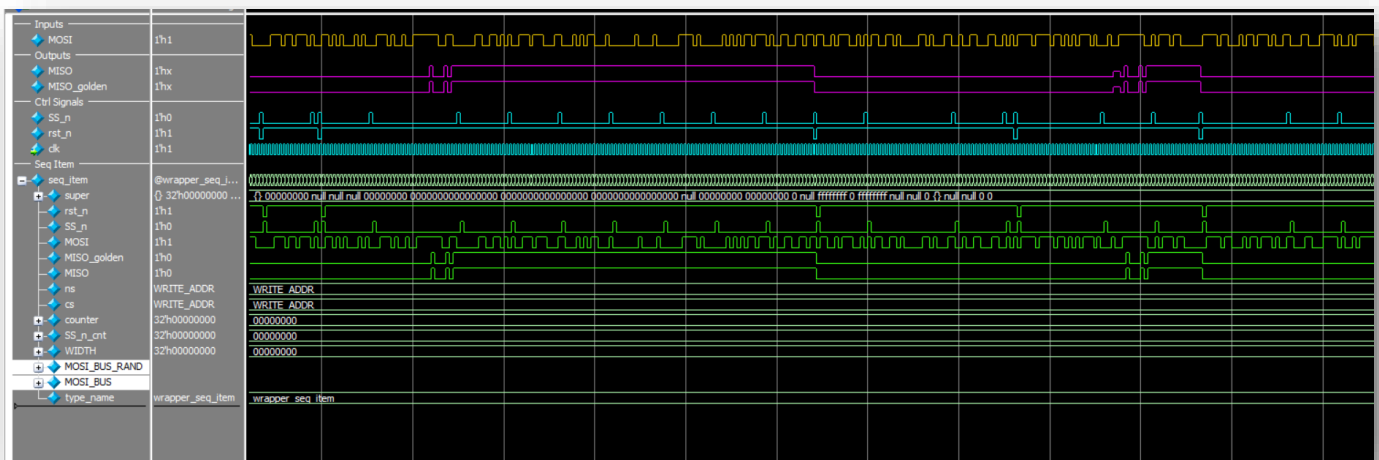
Write only snippet:



Read only snippet:



Write / read snippet:



Transcript:

```

# UVM_INFO ../uvm/11-wrapper_test.sv(88) @ 2: uvm_test_top [run_phase] Reset Deasserted
# UVM_INFO ../uvm/11-wrapper_test.sv(90) @ 2: uvm_test_top [run_phase] Write only Sequencer Begin
# UVM_INFO ../uvm/11-wrapper_test.sv(92) @ 2002: uvm_test_top [run_phase] Write only Sequencer end
# UVM_INFO ../uvm/11-wrapper_test.sv(94) @ 2002: uvm_test_top [run_phase] Read only Sequencer Begin
# UVM_INFO ../uvm/11-wrapper_test.sv(96) @ 4002: uvm_test_top [run_phase] Read only Sequencer end
# UVM_INFO ../uvm/11-wrapper_test.sv(98) @ 4002: uvm_test_top [run_phase] Write & Read Sequencer Begin
# UVM_INFO ../uvm/11-wrapper_test.sv(100) @ 6002: uvm_test_top [run_phase] Write & Read Sequencer end
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 6002: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO ../uvm/09-ram_scoreboard.sv(55) @ 6002: uvm_test_top.my_ram_env.sb [report_phase] RAM: Total successful counts:3001
# UVM_INFO ../uvm/09-ram_scoreboard.sv(56) @ 6002: uvm_test_top.my_ram_env.sb [report_phase] RAM: Total failed counts:0
# UVM_INFO ../uvm/09-SPI_scoreboard.sv(49) @ 6002: uvm_test_top.my_slave_env.sb [report_phase] Slave: Total Successful Transactions: 3001
# UVM_INFO ../uvm/09-SPI_scoreboard.sv(50) @ 6002: uvm_test_top.my_slave_env.sb [report_phase] Slave: Total Failed Transactions: 0
# UVM_INFO ../uvm/09-wrapper_scoreboard.sv(50) @ 6002: uvm_test_top.my_wrapper_env.sb [report_phase] Wrapper: total successful transactions: 3001
# UVM_INFO ../uvm/09-wrapper_scoreboard.sv(51) @ 6002: uvm_test_top.my_wrapper_env.sb [report_phase] Wrapper: total Failed transactions: 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 18
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RNTSTI] 1
# [TEST_DONE] 1
# [report_phase] 6
# [run_phase] 8
# ** Note: $finish : D:/Mina/Program_Files/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 6002 ns Iteration: 61 Instance: /top

```

17. Assertions table

Feature	Assertions
Whenever the rst is asserted, MISO is low	@(posedge clk) (!rst_n -> ~MISO)
the MISO remains with a stable value eventually as long as it is not a read data operation	@(posedge clk) disable iff (~rst_n) !tx_valid => (MISO == \$stable(MISO) [->1]);